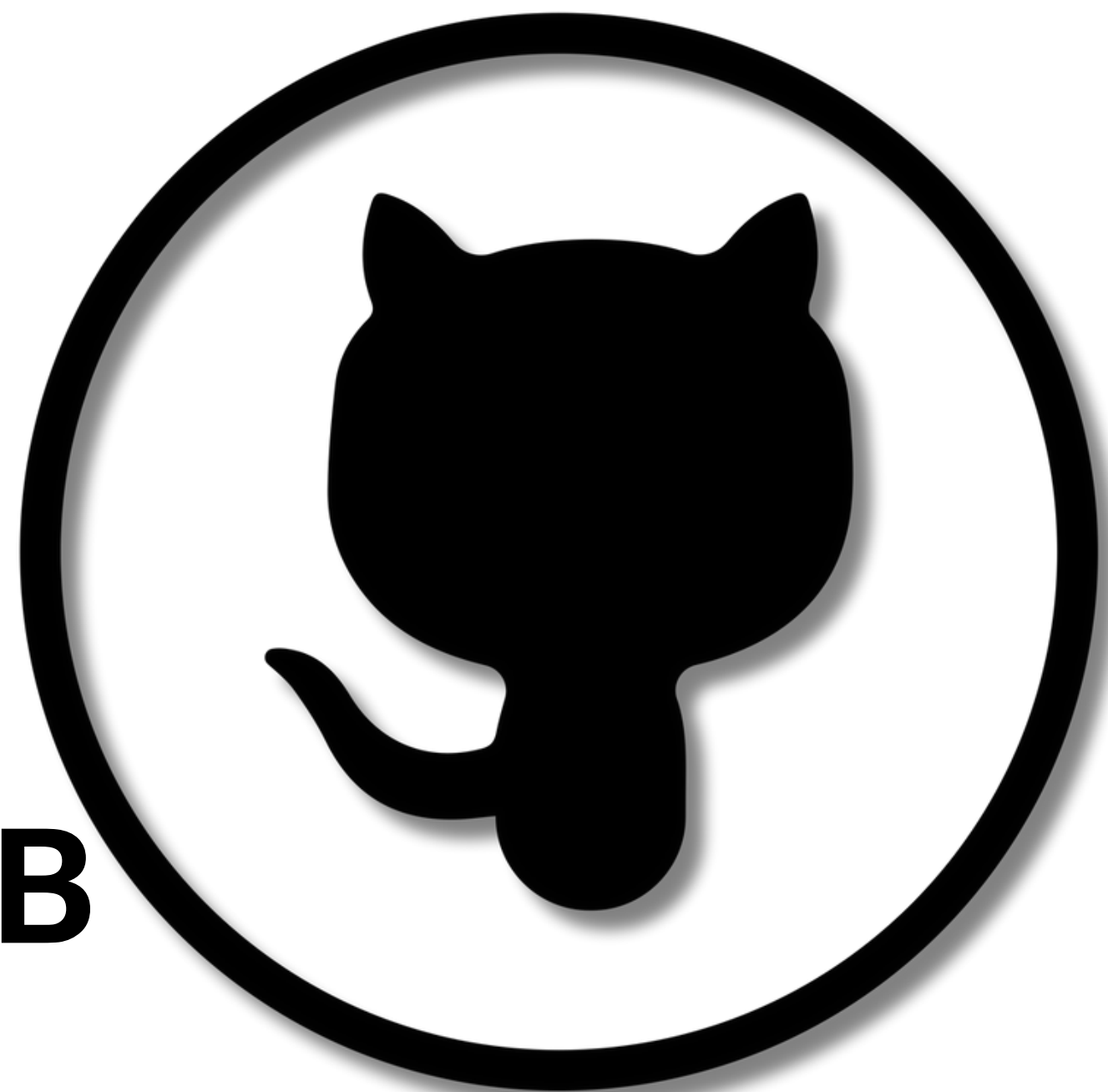




!=

GIT - GITHUB





O QUE É GIT?



O Git é um sistema de controle de versão distribuído, o que significa que um clone local do projeto é um repositório de controle de versão completo.

Esses repositórios locais totalmente funcionais facilitam o trabalho offline ou remoto. Os desenvolvedores confirmam o trabalho localmente e depois sincronizam a cópia do repositório com a cópia no servidor. Esse paradigma é diferente do controle de versão centralizado, no qual os clientes devem sincronizar o código com um servidor antes de criar novas versões do código.

LÓGICA

SOBRE **CONTROLE** DE VERSÃO

LÓGICA

Software capaz de gerenciar
versões de qualquer tipo de
arquivo.



SOBRE **CONTROLE** DE VERSÃO

LÓGICA

Software capaz de **gerenciar**
versões de qualquer tipo de
arquivo.

Mais usado durante
desenvolvimento de software.



SOBRE **CONTROLE** DE VERSÃO

LÓGICA

Software capaz de **gerenciar versões** de qualquer tipo de arquivo.

Mais usado durante **desenvolvimento de software**.

SOBRE **CONTROLE** DE VERSÃO

Tem a oportunidade de acompanhar o “**Histórico de alterações**”.

LÓGICA

Software capaz de **gerenciar versões** de qualquer tipo de arquivo.

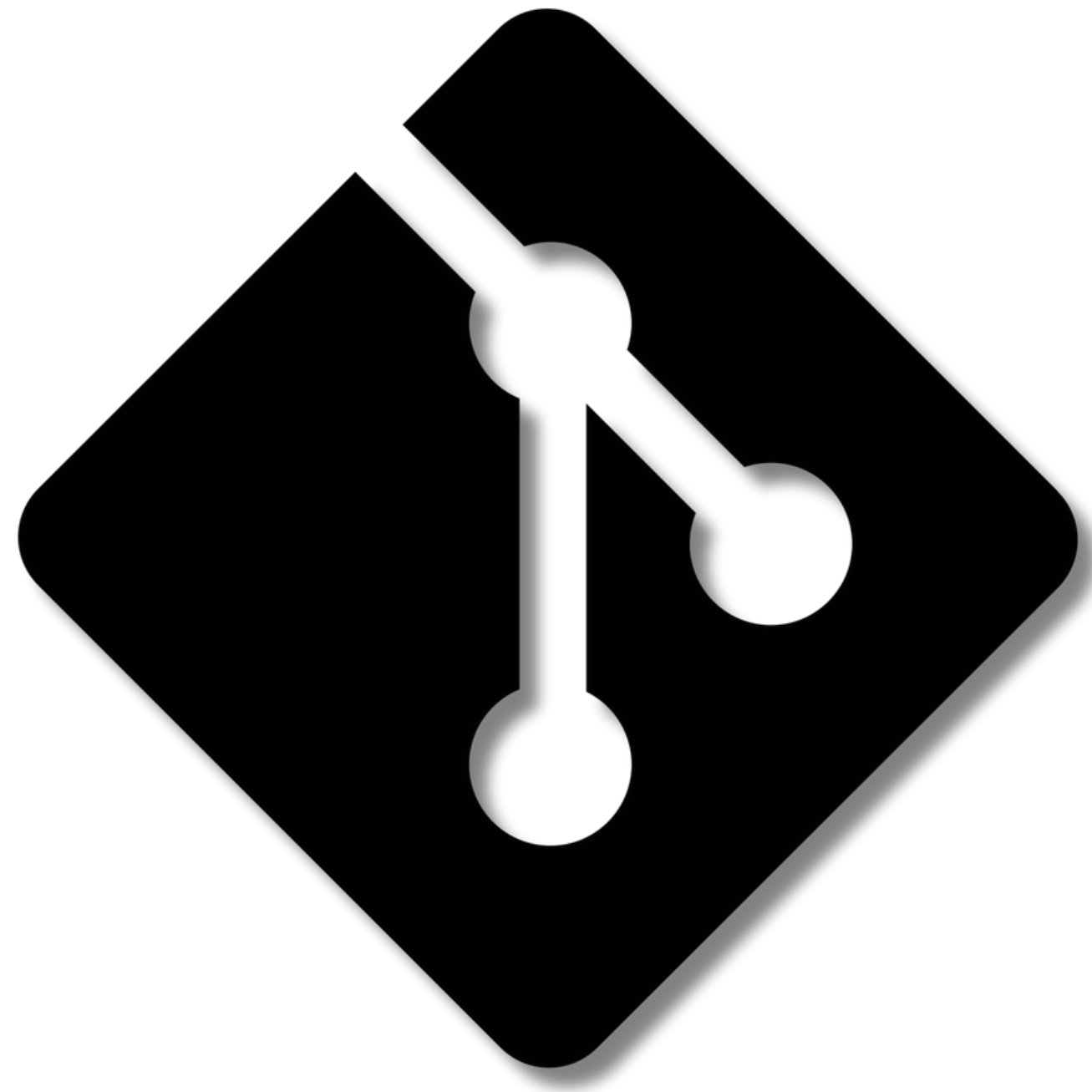
Mais usado durante **desenvolvimento de software**.

SOBRE **CONTROLE** DE VERSÃO

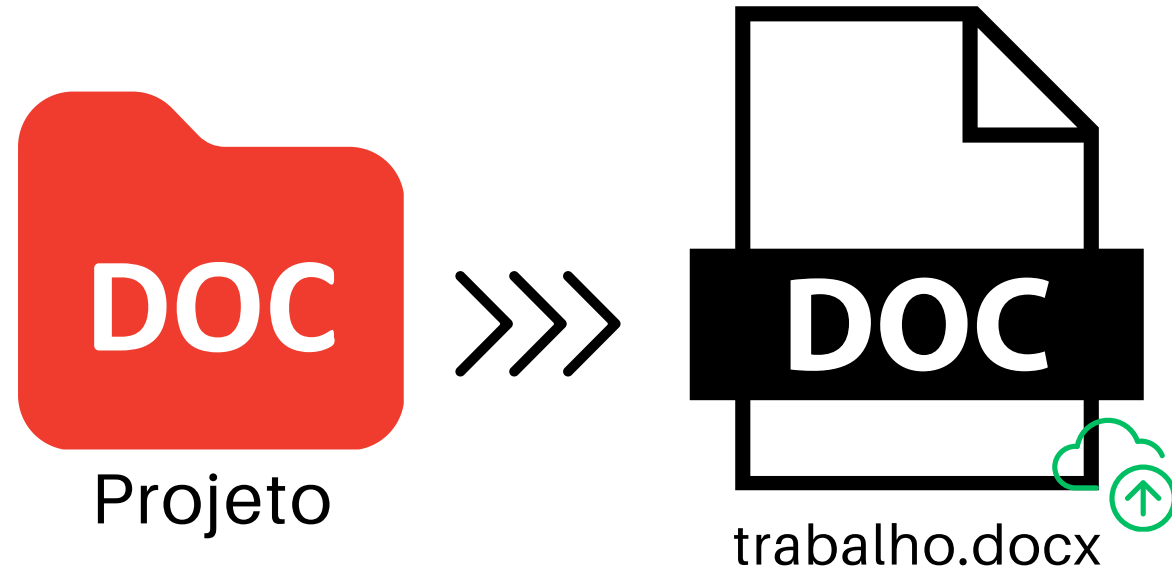
Tem a oportunidade de acompanhar o “**Histórico de alterações**”.

Podemos utilizá-los **pequenos ou grandes projetos**.

Como o versionamento acontece?



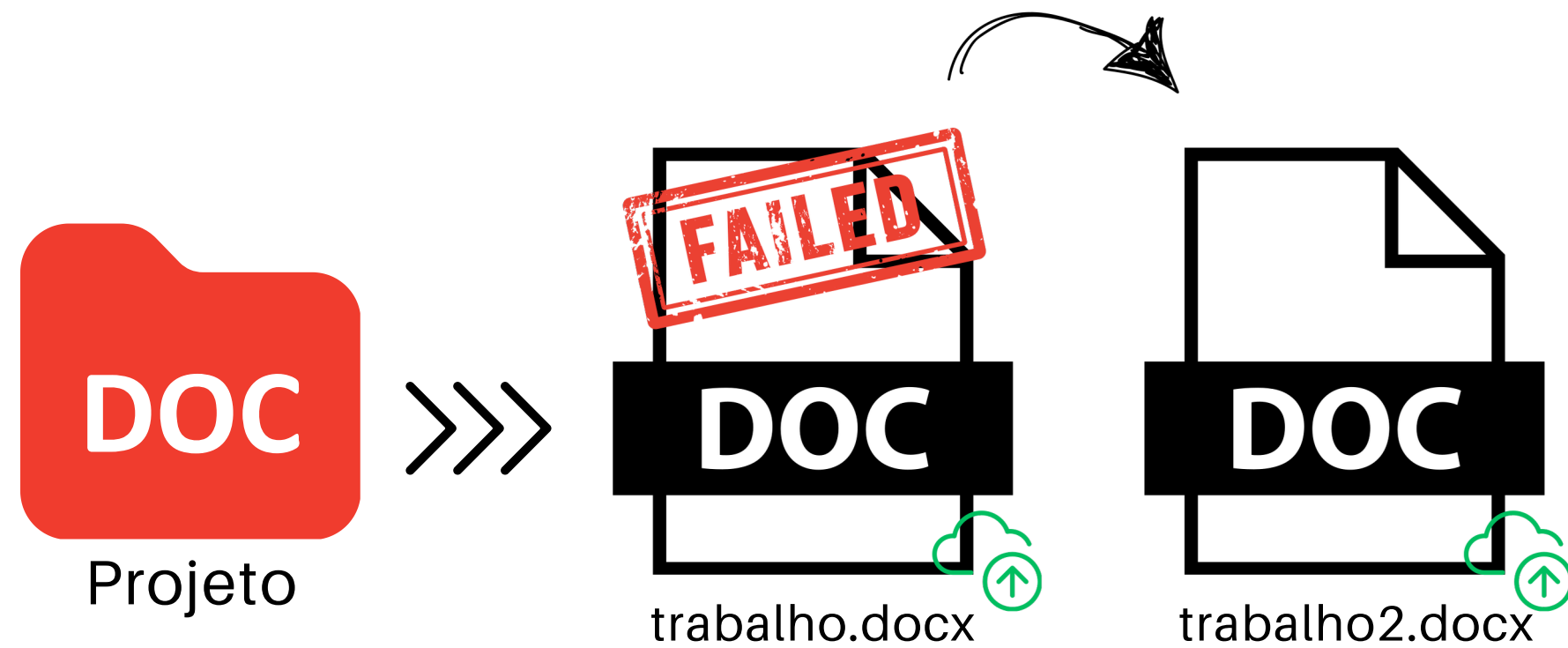
ALTERNATIVA



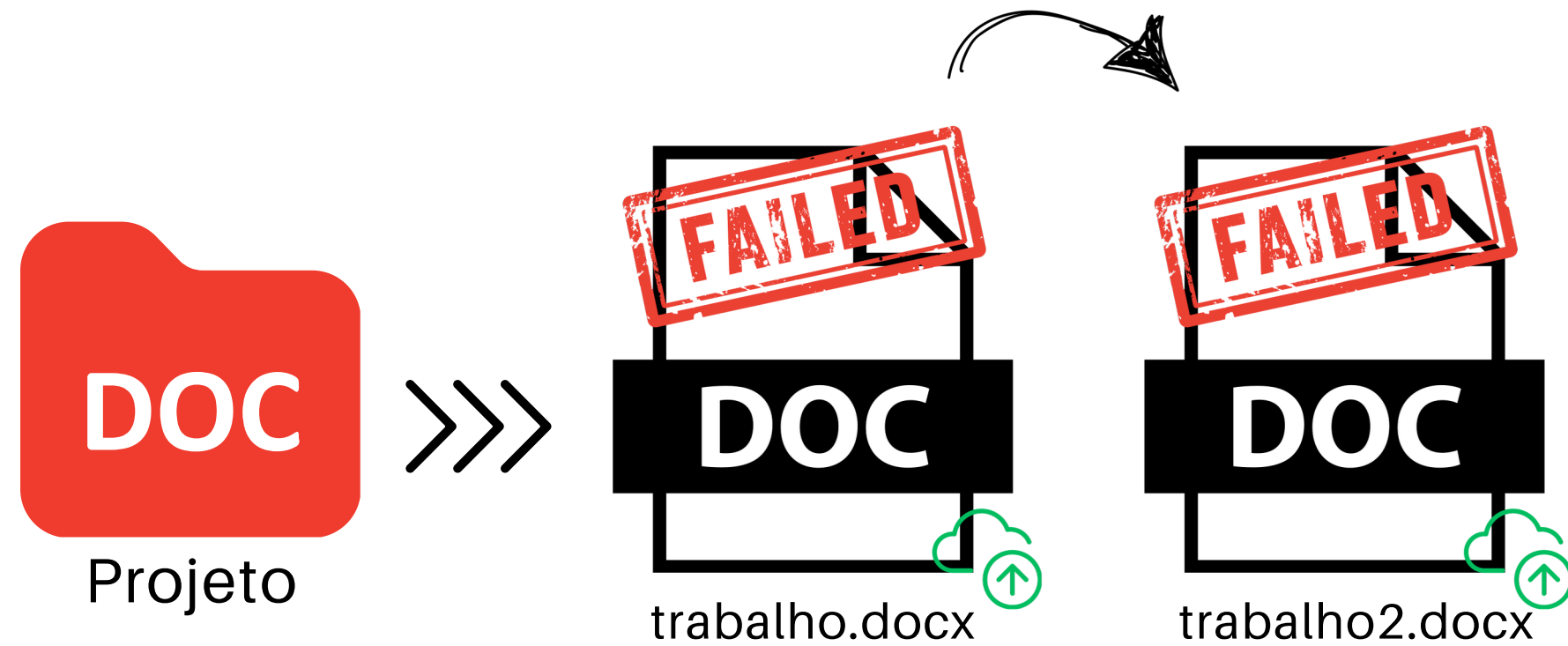
ALTERNATIVA



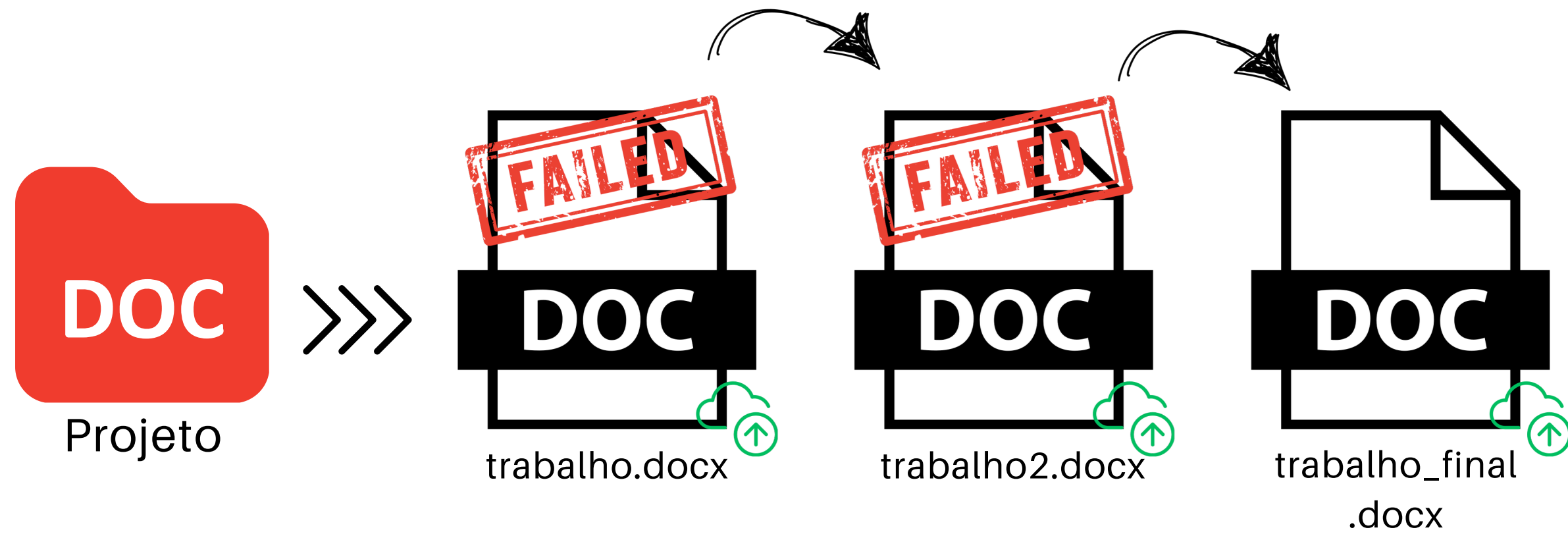
ALTERNATIVA



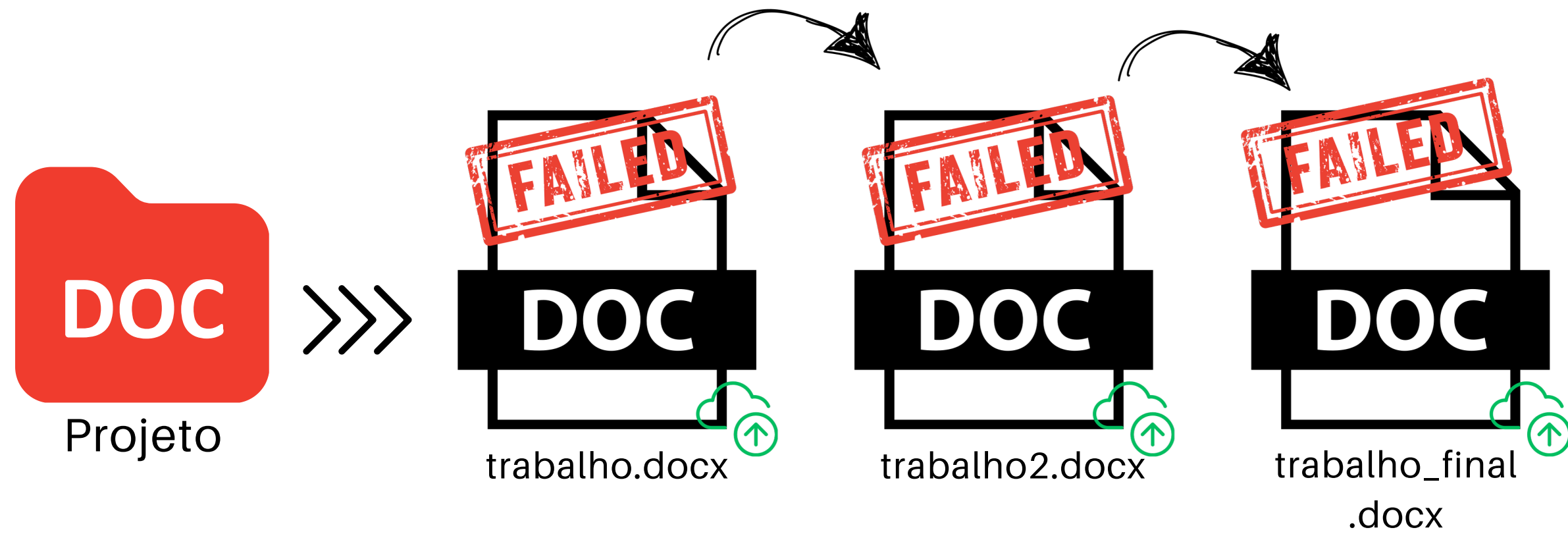
ALTERNATIVA



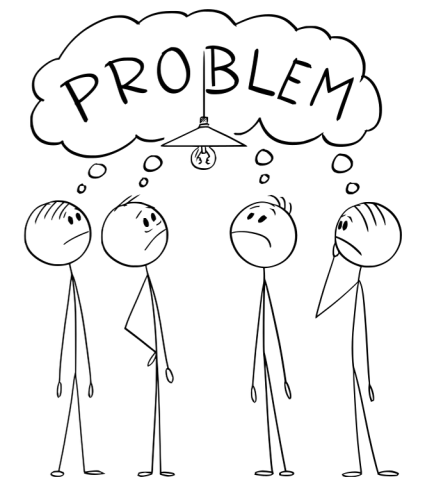
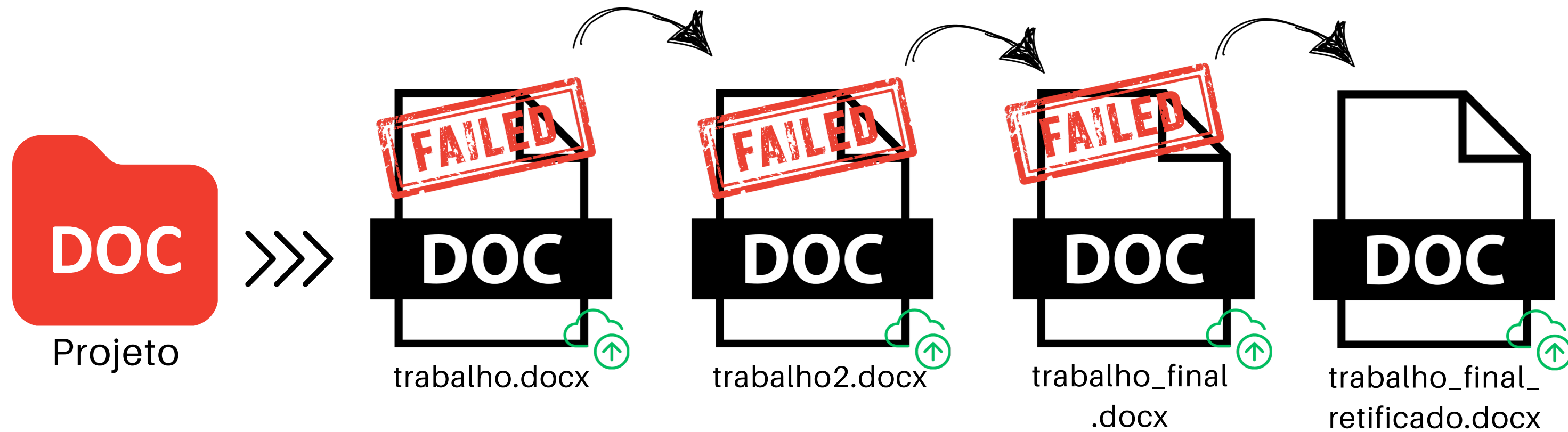
ALTERNATIVA



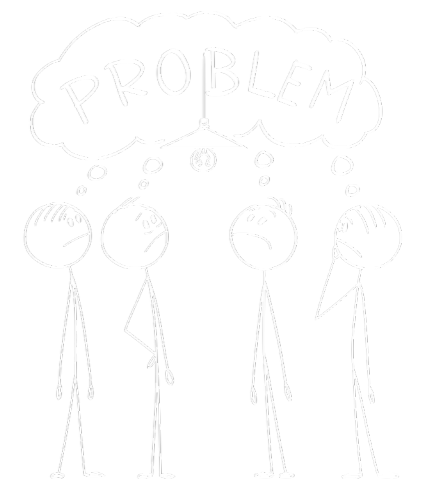
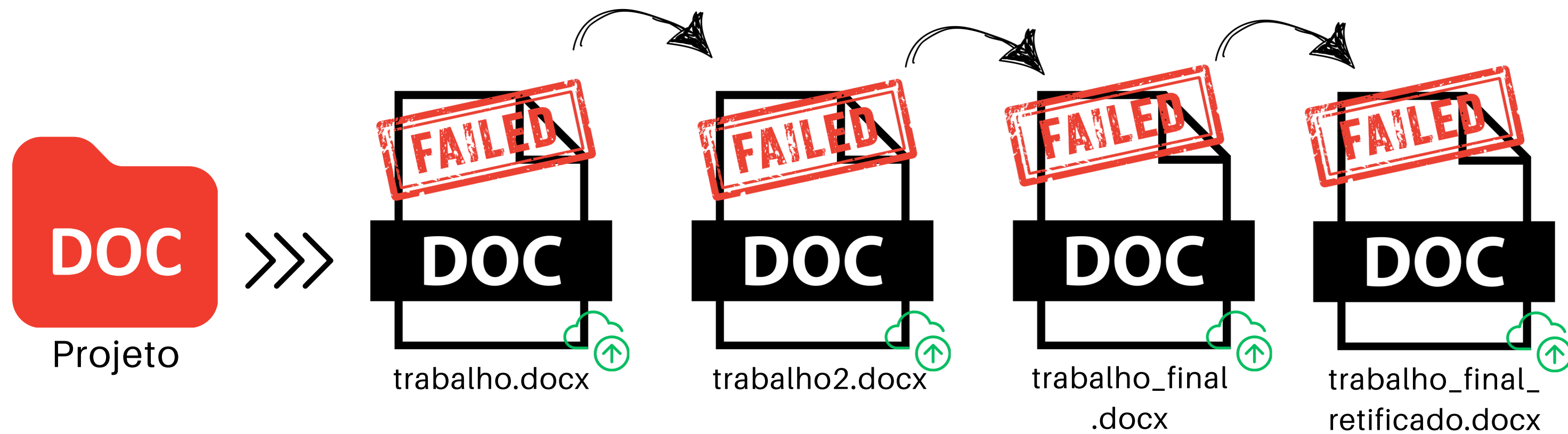
ALTERNATIVA



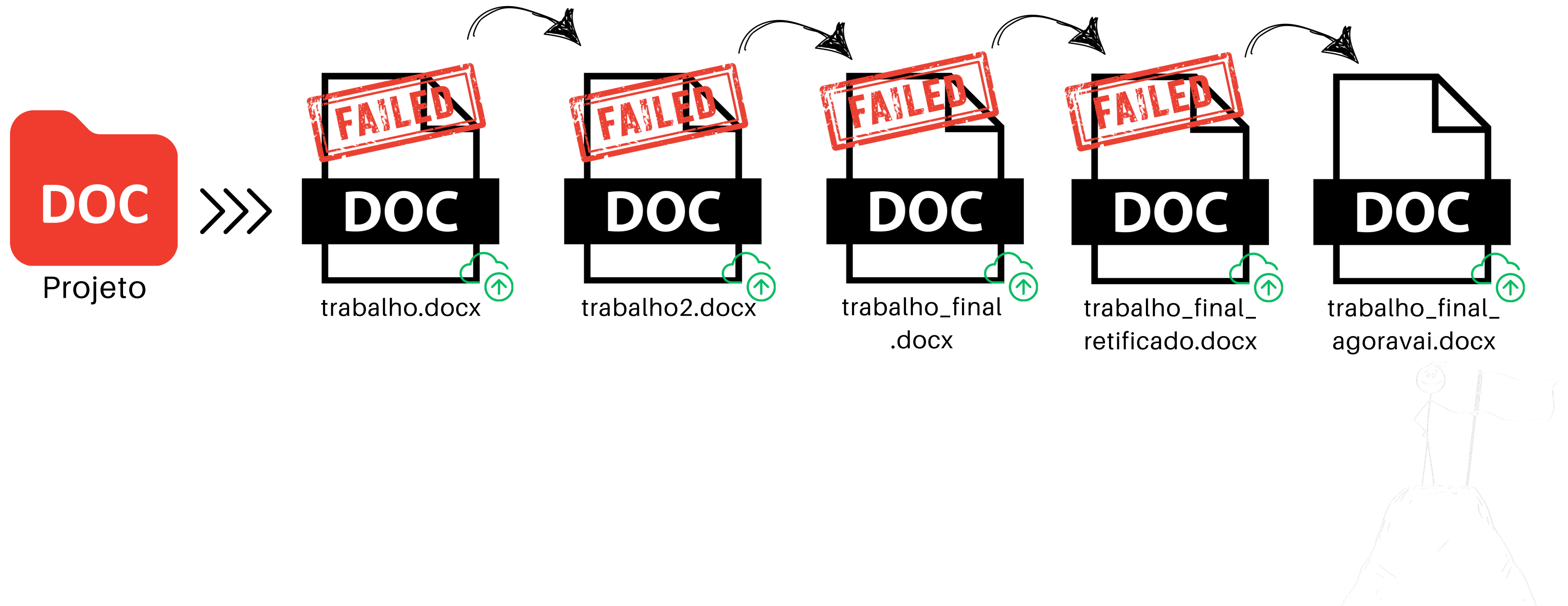
ALTERNATIVA



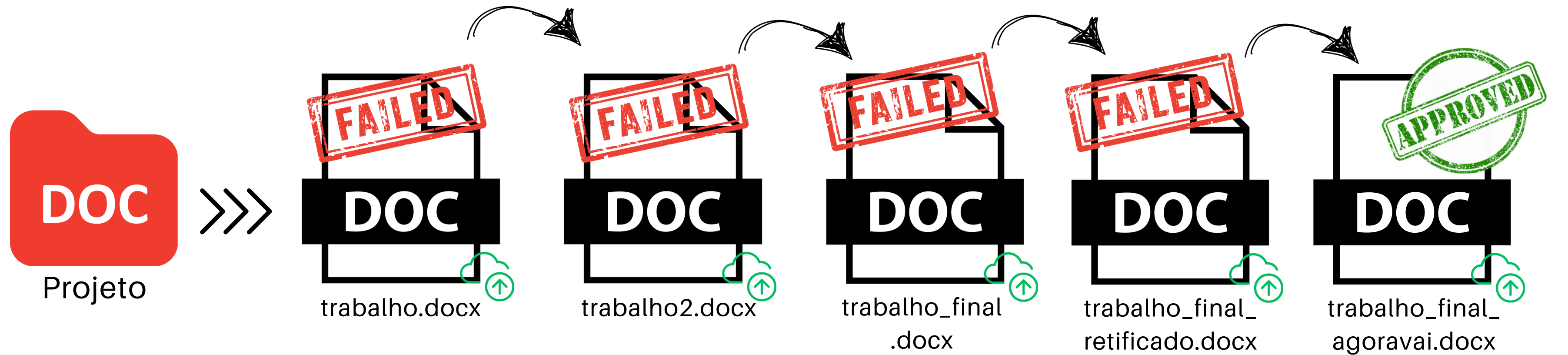
ALTERNATIVA



ALTERNATIVA

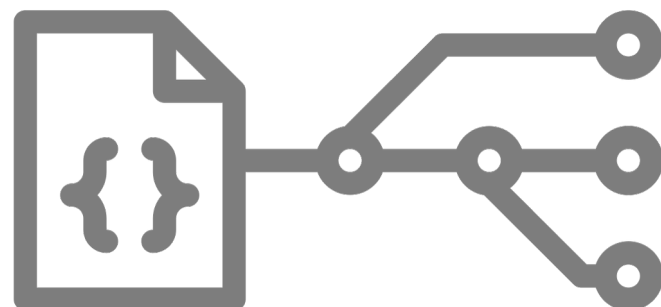


ALTERNATIVA

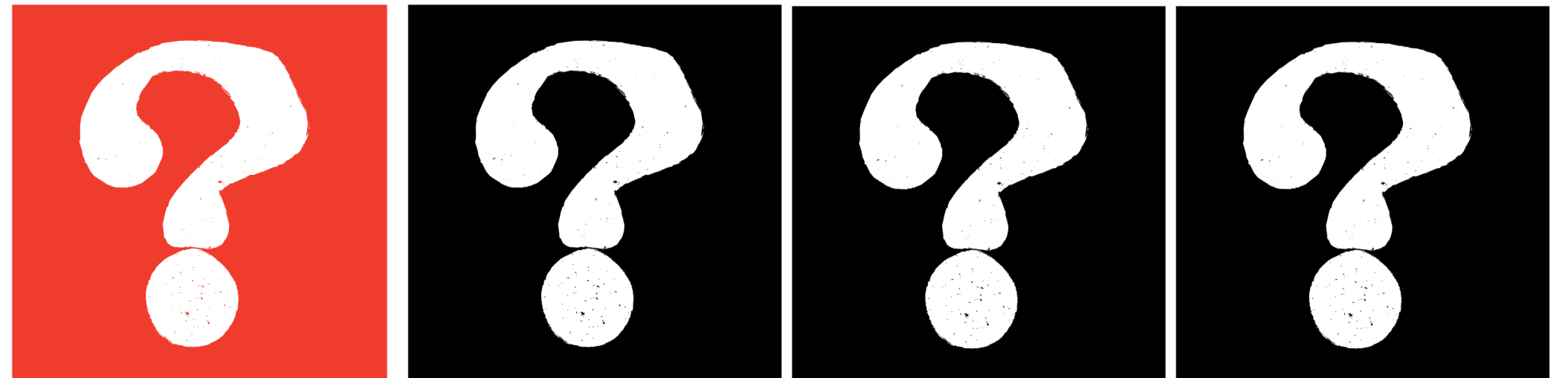


A low-angle, black and white photograph of several tall skyscrapers reaching towards a cloudy sky. The perspective is from the ground looking up, making the buildings appear to converge towards the top of the frame. The image has a torn paper edge effect on the right side, separating it from the text.

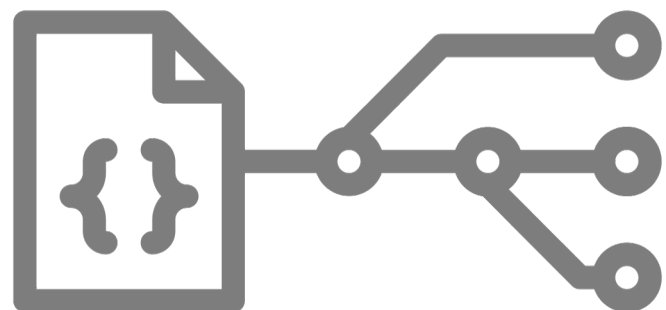
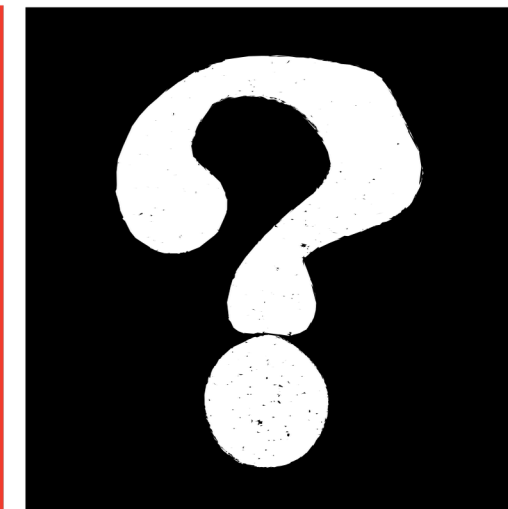
EXEMPLOS DE EMPRESAS QUE UTILIZAM O CONTROLE DE VERSÃO...



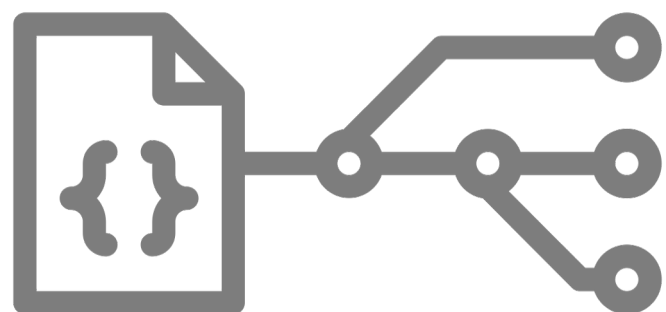
android

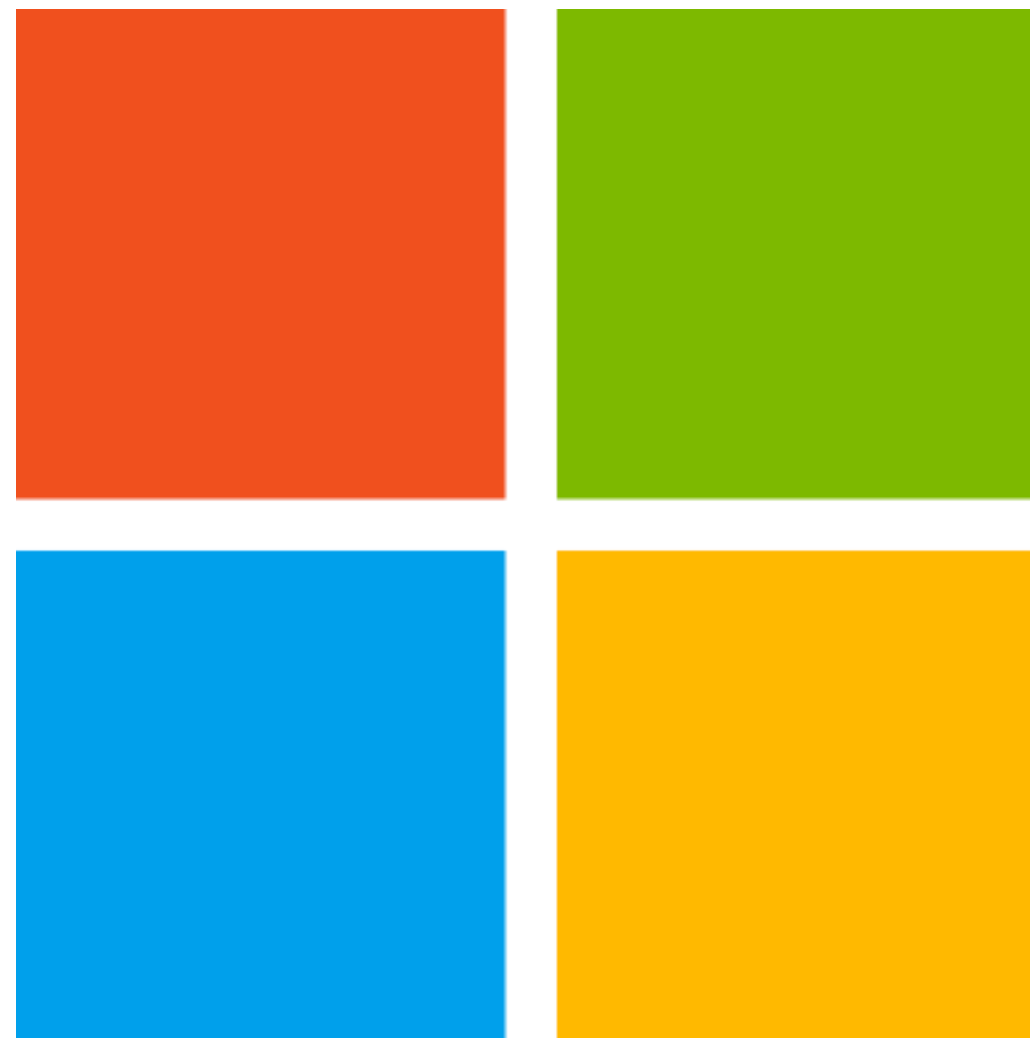


NETFLIX

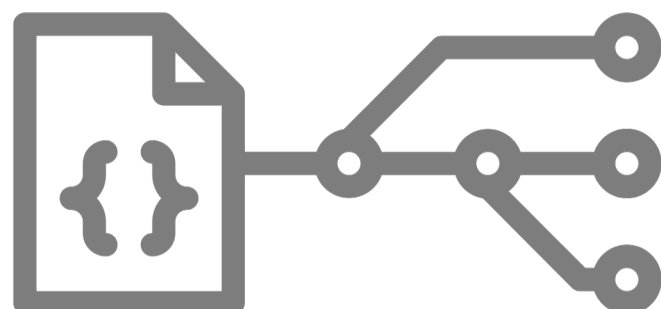


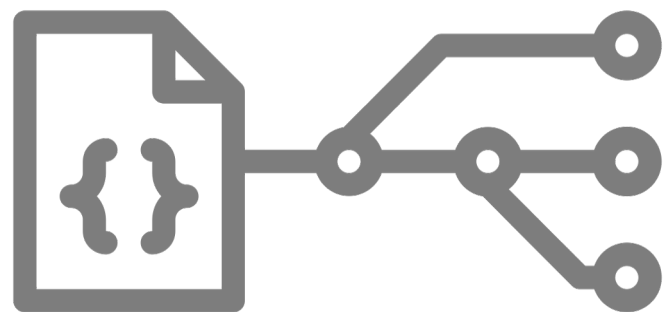
GOOGLE





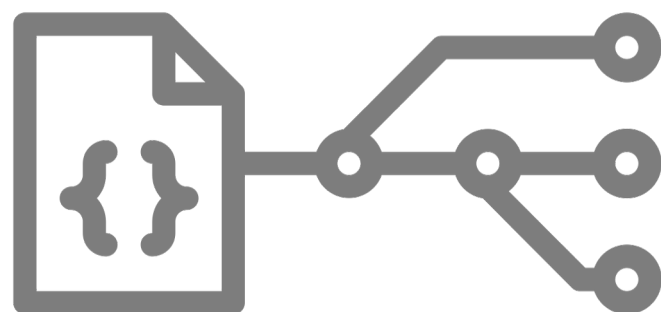
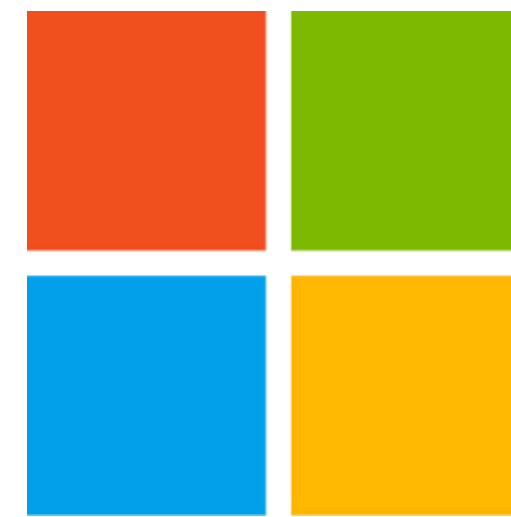
Microsoft





FACEBOOK

Prof. Carlos Sabino



Funcionalidades



HISTÓRICO



FRENTES DE TRABALHO



RASTREABILIDADE



RAMOS (BRANCHES)

Funcionalidades



HISTÓRICO

Funcionalidades



HISTÓRICO

└ Alternar entre versões.

Funcionalidades



HISTÓRICO

- └ Alternar entre versões.
- └ Restaurar versões antigas caso algum problema aconteça.

Funcionalidades



HISTÓRICO

- └ Alternar entre versões.
- └ Restaurar versões antigas caso algum problema aconteça.



FRENTES DE TRABALHO

Funcionalidades



HISTÓRICO

- └ Alternar entre versões.
- └ Restaurar versões antigas caso algum problema aconteça.



FRENTES DE TRABALHO

- └ Desenvolvimento paralelo no mesmo projeto.

Funcionalidades



HISTÓRICO

- └ Alternar entre versões.
- └ Restaurar versões antigas caso algum problema aconteça.



FRENTES DE TRABALHO

- └ Desenvolvimento paralelo no mesmo projeto.



RASTREABILIDADE

Funcionalidades



HISTÓRICO

- └ Alternar entre versões.
- └ Restaurar versões antigas caso algum problema aconteça.



FRENTES DE TRABALHO

- └ Desenvolvimento paralelo no mesmo projeto.



RASTREABILIDADE

- └ Identificar quando uma ou mais mudanças foram executadas.

Funcionalidades



HISTÓRICO

- └ Alternar entre versões.
- └ Restaurar versões antigas caso algum problema aconteça.



RASTREABILIDADE

- └ Identificar quando uma ou mais mudanças foram executadas.



FRENTES DE TRABALHO

- └ Desenvolvimento paralelo no mesmo projeto.



RAMOS (BRANCHES)

Funcionalidades



HISTÓRICO

- └ Alternar entre versões.
- └ Restaurar versões antigas caso algum problema aconteça.



RASTREABILIDADE

- └ Identificar quando uma ou mais mudanças foram executadas.



FRENTES DE TRABALHO

- └ Desenvolvimento paralelo no mesmo projeto.



RAMOS (BRANCHES)

- └ Possibilidade de criar diferentes versões a partir de um único ponto de partida

Funcionalidades



HISTÓRICO

- └ Alternar entre versões.
- └ Restaurar versões antigas caso algum problema aconteça.



RASTREABILIDADE

- └ Identificar quando uma ou mais mudanças foram executadas.



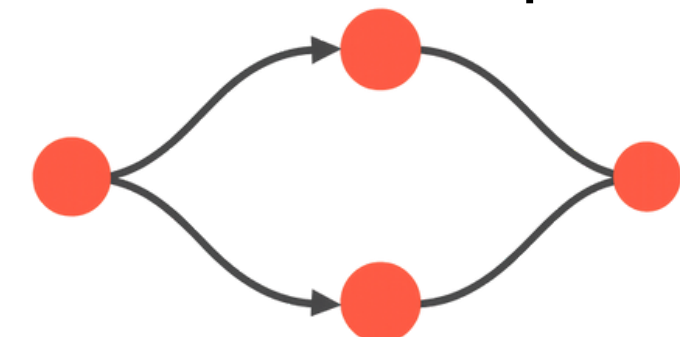
FRENTES DE TRABALHO

- └ Desenvolvimento paralelo no mesmo projeto.



RAMOS (BRANCHES)

- └ Possibilidade de criar diferentes versões a partir de um único ponto de partida
- └ Mesclar funcionalidades após testadas





Tipos de controle de versão...

Centralizado



ESTRUTURA

1. Servidor contém todo o histórico
2. Padrão por muitos anos

- *Subversion (SVN) criado em 2000*
- *Git criado no final de 2005*

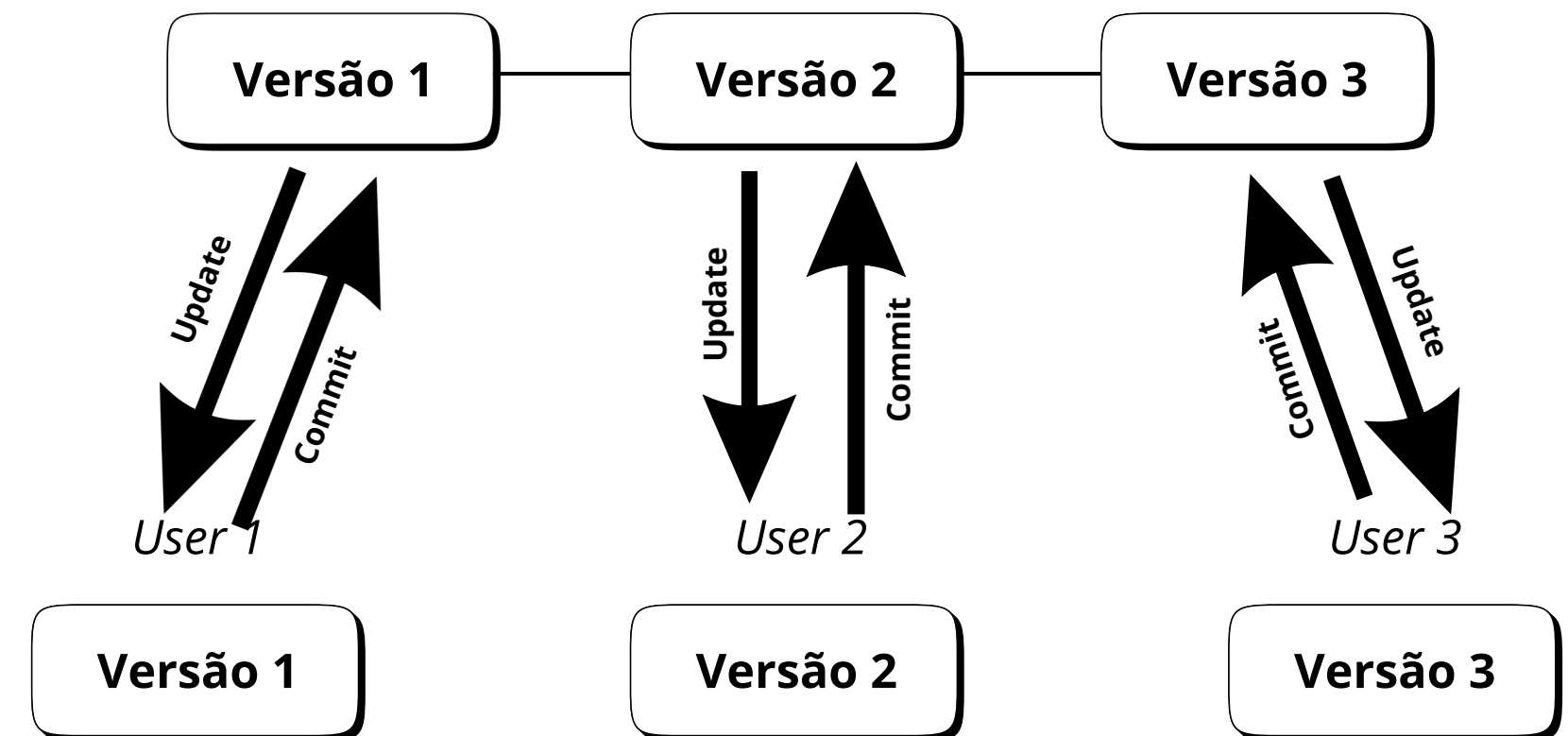


DESVANTAGEM

1. Servidor contém todo o histórico

- *Qualquer problema no servidor impacta o projeto*

SERVIDOR



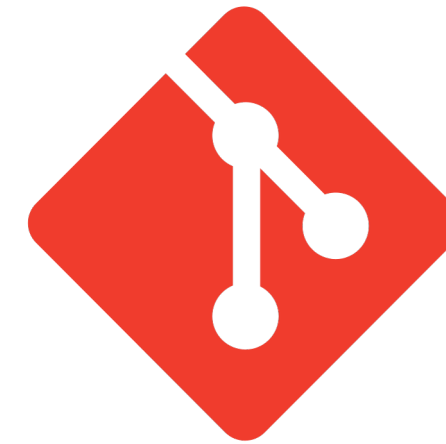
PERFORCE

Distribuído

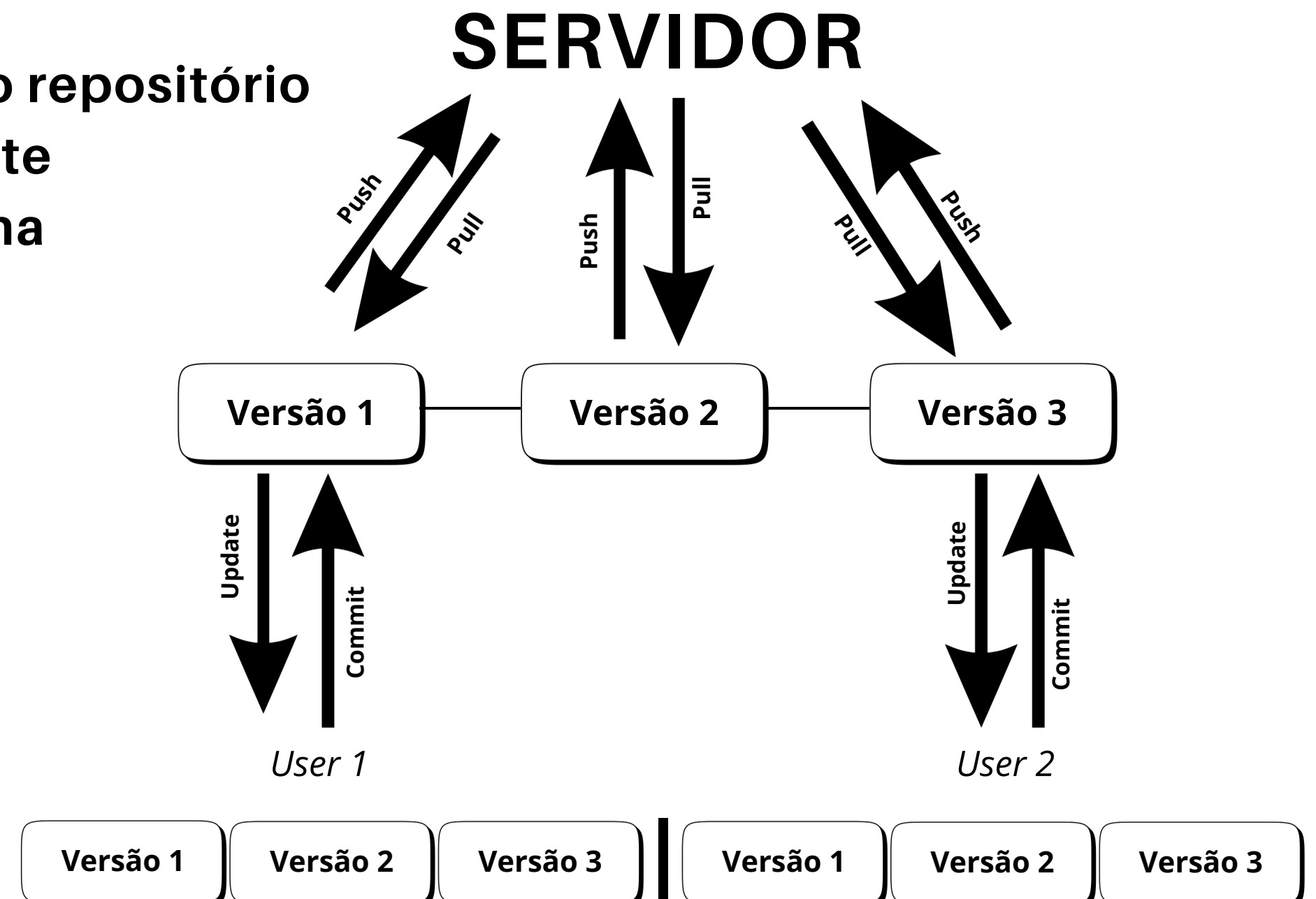


ESTRUTURA

- 1. Toda máquina possui uma cópia do repositório
- 2. Mudanças são realizadas localmente
- 3. Não depende de uma única máquina



git





AÇÕES LOCAIS

- 1. Navegação de histórico
- 2. Criação de branch

DESENVOLVIMENTO NÃO LINEAR

- 1. Várias versões a partir de um único ponto
- 2. Facilidade para diversas pessoas realizarem alterações sem dependência de arquivos

INTEGRIDADE

- 1. Uma vez que um arquivo é adicionado, todo seu histórico é guardado.
- 2. Mesmo a remoção de arquivo é feito através de uma adição de versão onde diz que o arquivo foi removido

INDEPENDÊNCIA DA COMPLEXIDADE DO PROJETO

- 1. Eficiência para pequenos ou grandes projetos
- 2. Complexidade de uso permanece a mesma



Instalação e configuração inicial...

Crie e lance software na melhor plataforma

Junte-se à plataforma de desenvolvimento usada no mercado

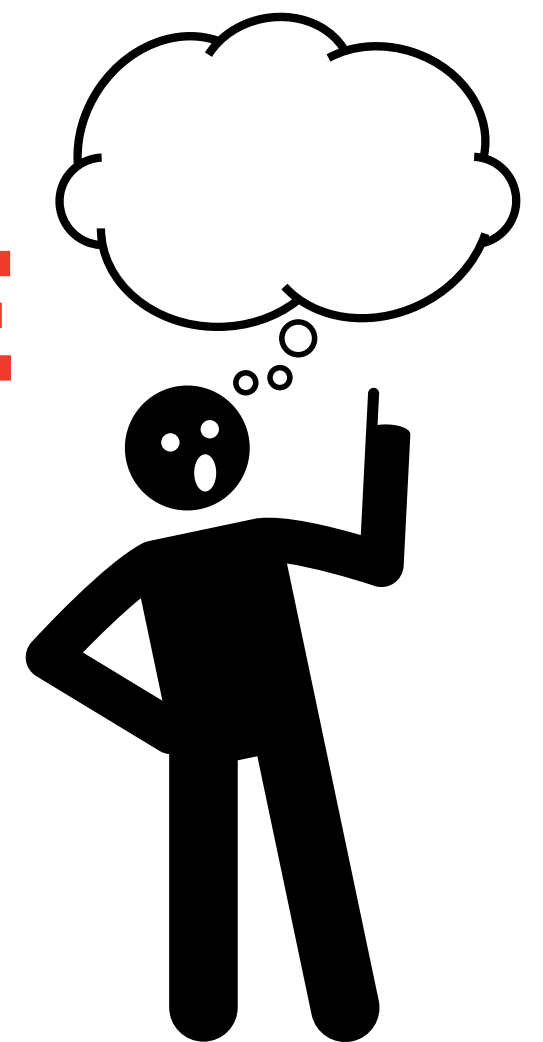
Digite seu e-mail



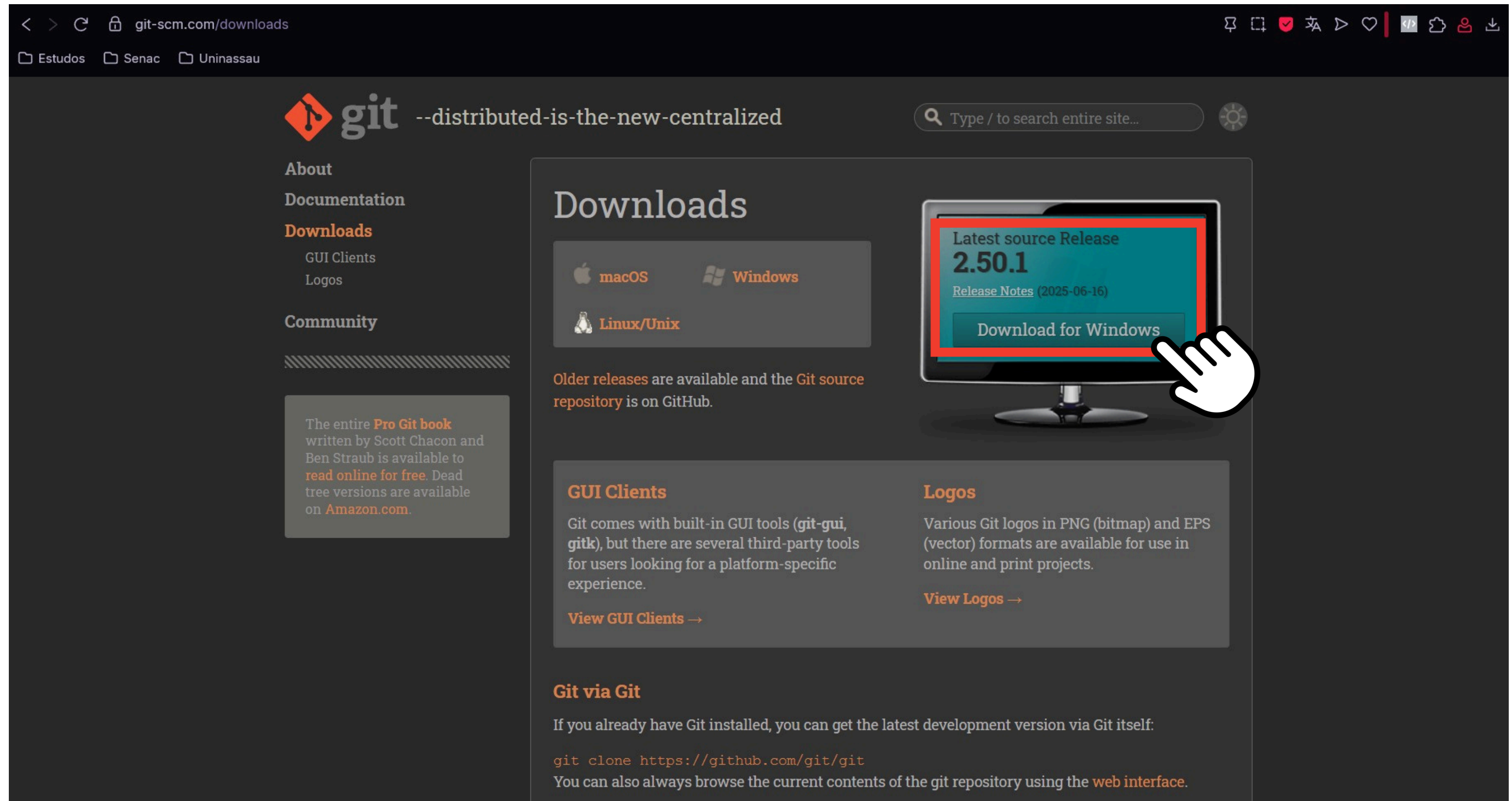
Cadastrar-se no GitHub

Prof. Carlos

**SE AINDA NÃO
TEM CONTA NO
GITHUB,
LEMBRA DE
CRIAR...**



Instalação



Instalação

git --fast-version-control

Search: Type / to search entire site...

Download for Windows

[Click here to download](#) the latest (2.50.1) x64 version of Git for Windows. This is the most recent maintained build. It was released 35 days ago, on 2025-07-08.

Other Git for Windows downloads

- Standalone Installer
- [Git for Windows/x64 Setup.](#)
- [Git for Windows/ARM64 Setup.](#)
- Portable ("thumbdrive edition")
- [Git for Windows/x64 Portable.](#)
- [Git for Windows/ARM64 Portable.](#)

Using winget tool

Install [winget tool](#) if you don't already have it, then type this command in command prompt or Powershell.

```
winget install --id Git.Git -e --source winget
```

The current source code release is version 2.50.1. If you want the newer version, you can build it from [the source code](#).

Now What?

Now that you have downloaded Git, it's time to start using it.

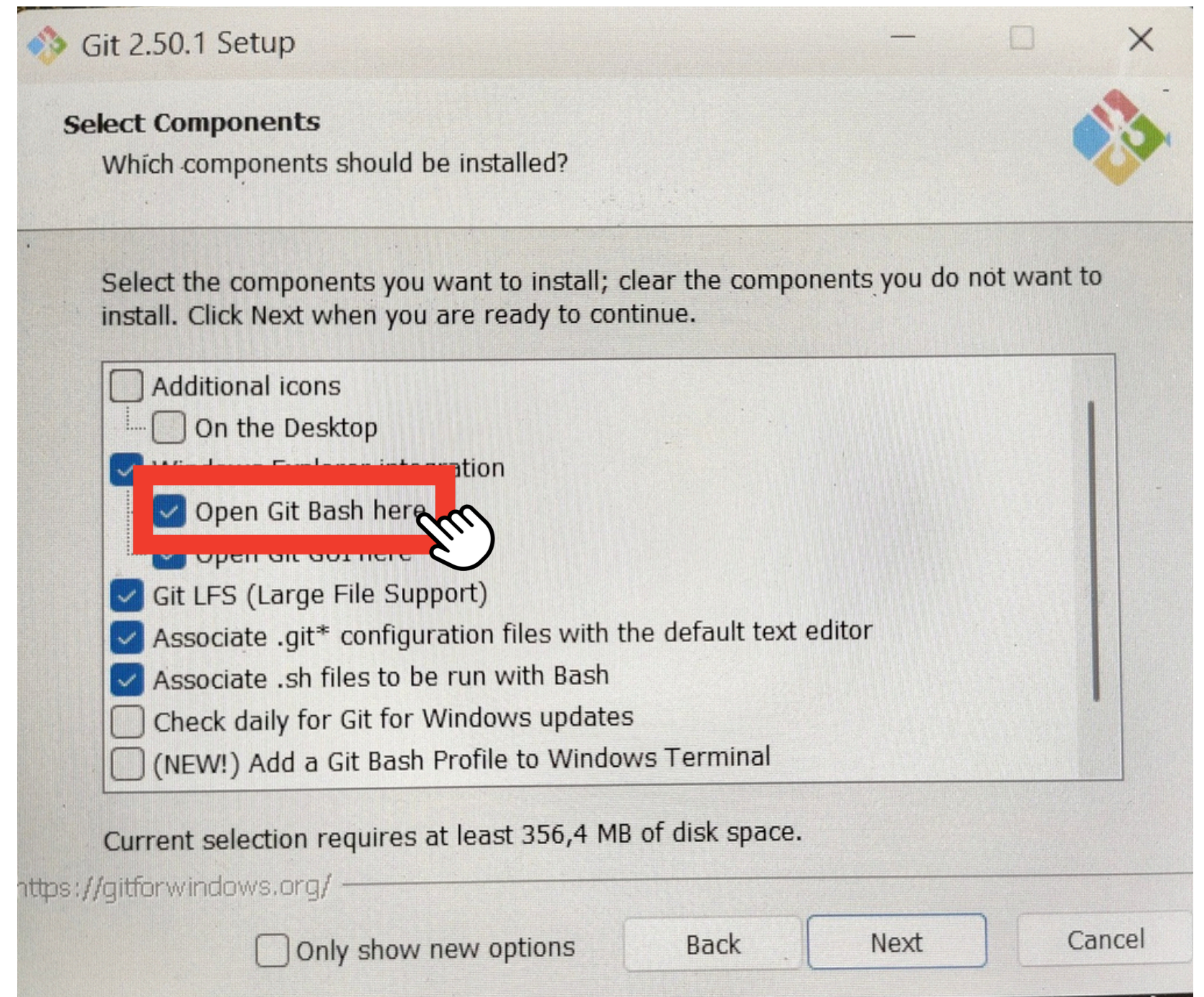
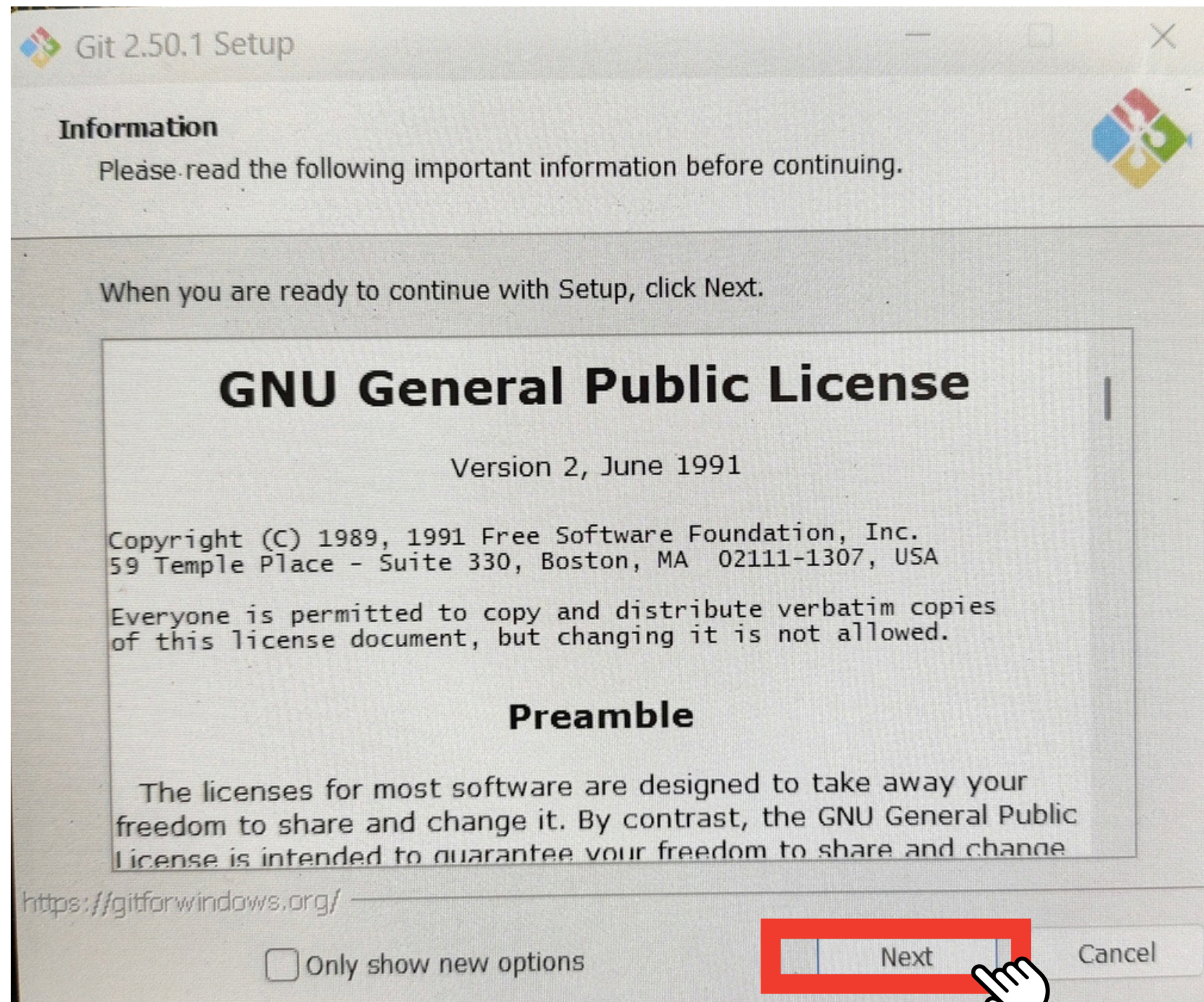
Left sidebar:

- About
- Documentation
- Downloads**
 - GUI Clients
 - Logos
- Community

Bottom left box:

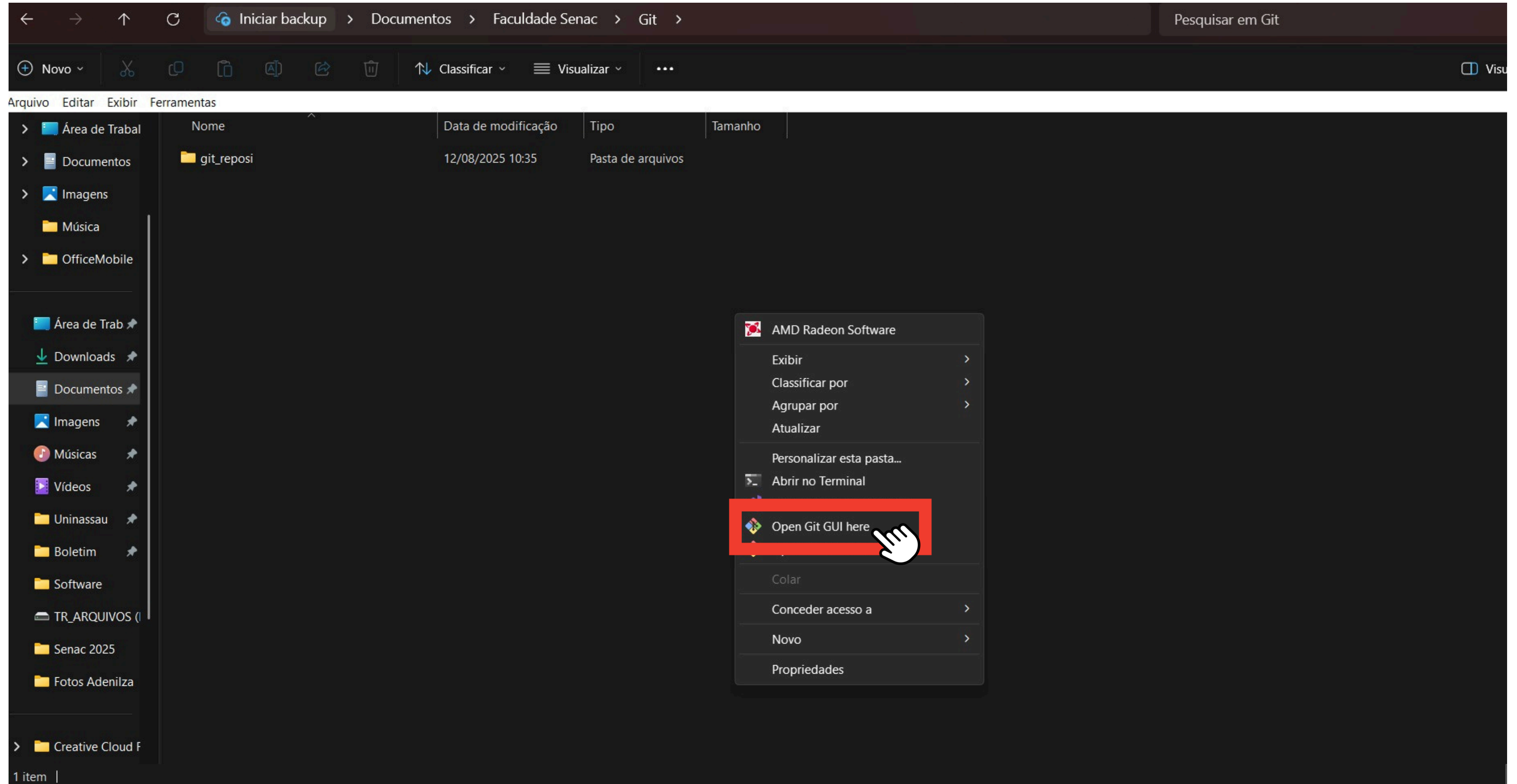
The entire [Pro Git book](#) written by Scott Chacon and Ben Straub is available to [read online for free](#). Dead tree versions are available on [Amazon.com](#).

Instalação

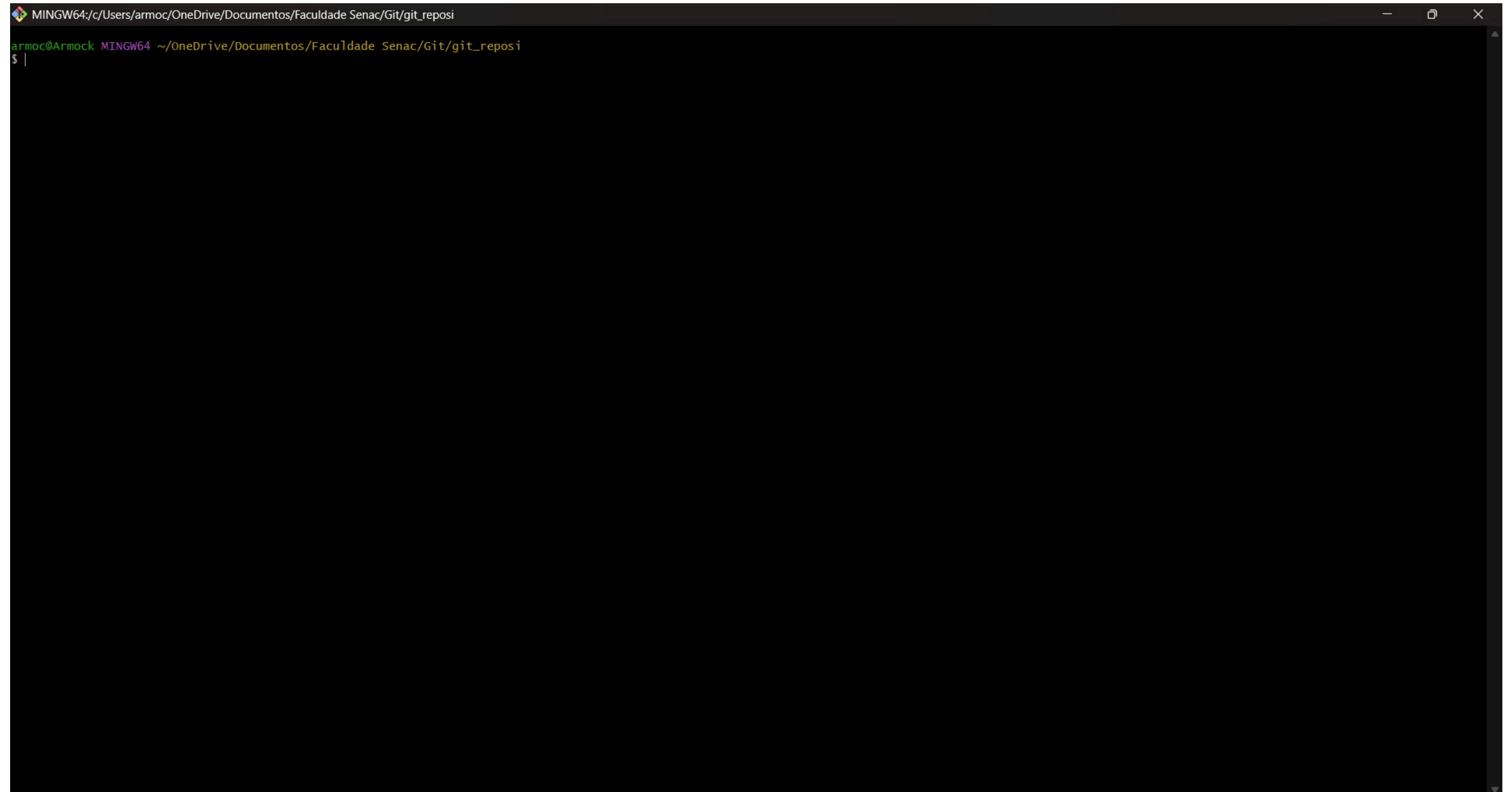


Depois dessa parte, avança até a instalação final...

Acessando o git



Acessando o git



```
MINGW64:/c:/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ |
```

Acessando o git

```
MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git/git_reposi  
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi  
$ |
```

Configuração de Usuário

 MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git/git_reposi

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.name "Thalyta Robertta Ramos Carmo"
```

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.email "professorarobertta@gmail.com"
```

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$
```

Configuração de Usuário - verificação

```
MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git/git_reposi

armoc@Armock MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.name "Thalyta Robertta Ramos Carmo"

armoc@Armock MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.email "professorarobertta@gmail.com"

armoc@Armock MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.name
Thalyta Robertta Ramos Carmo

armoc@Armock MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.email
professorarobertta@gmail.com

armoc@Armock MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$
```


Configuração de Usuário - limpando a interface

MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git/git_reposi

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.name "Thalyta Robertta Ramos Carmo"
```

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.email "professorarobertta@gmail.com"
```

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.name
Thalyta Robertta Ramos Carmo
```

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --global user.email
professorarobertta@gmail.com
```

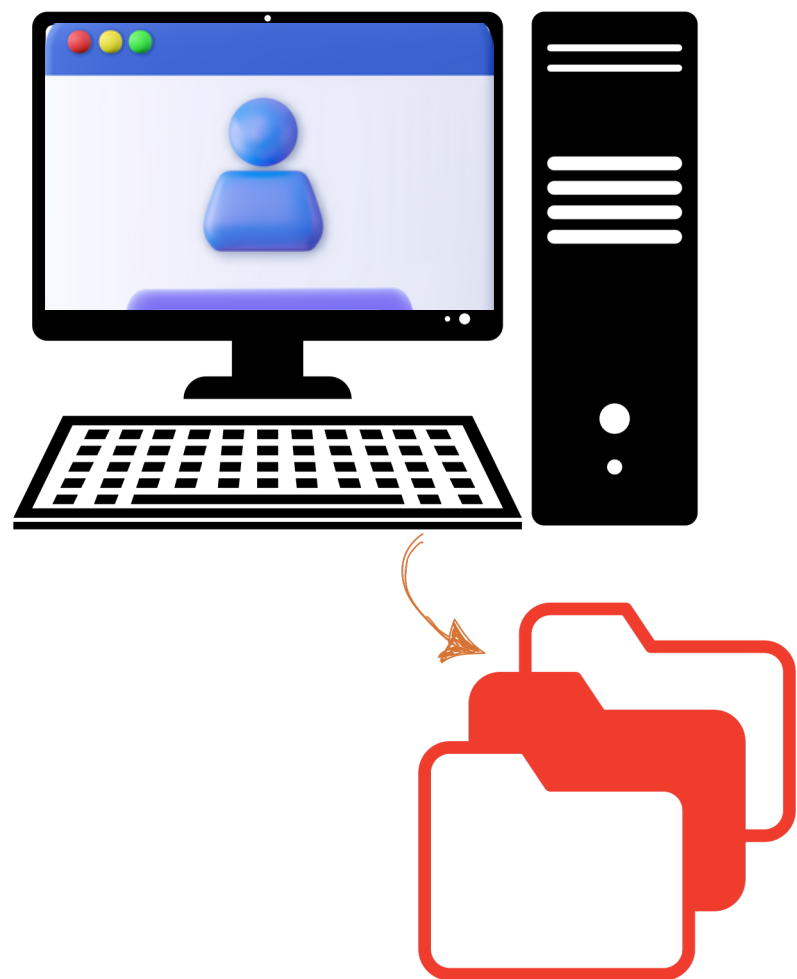
```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ clear
```

Configuração de Usuário - lista de configurações

```
MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Thalyta Robertta Ramos Carmo
user.email=professorarobertta@gmail.com
init.defaultbranch=master
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git/git_reposi
$
```

Próximos passos



Cria uma pasta
Crie um arquivo txt
Abra o Bash
Liste os arquivos: ls
deve aparecer o que acabou de criar

Criando repositórios

 MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git Repositor

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ ls
teste.txt
```

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ ls -a
./  ../  teste.txt
```

```
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ |
```

Criando repositórios

```
MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git Repositor

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ ls
teste.txt

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ ls -a
./ ../ teste.txt

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ git init
Initialized empty Git repository in C:/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git Repositor/.git/

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor (master)
$
```


Criando repositórios

```
MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git Repositor

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ ls
teste.txt

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ ls -a
./  ../  teste.txt

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ git init
Initialized empty Git repository in C:/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git Repositor/.git/

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor (master)
$ ls -a
./  ../  .git/  teste.txt

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor (master)
$ |
```


Mudando o nome da MASTER

```
MINGW64:/c/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git Repositor

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ ls
teste.txt

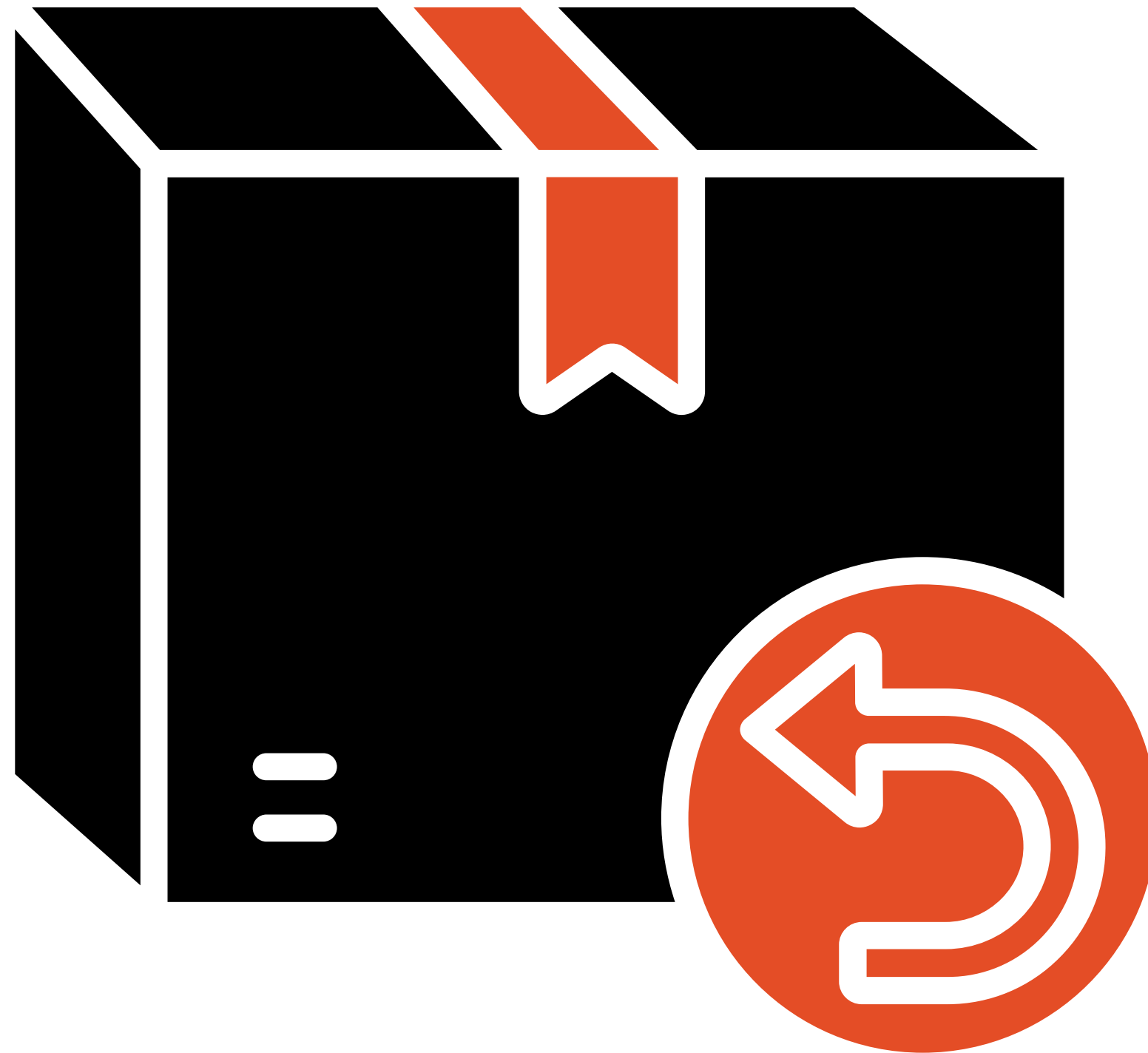
armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ ls -a
./  ../  teste.txt

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor
$ git init
Initialized empty Git repository in c:/Users/armoc/OneDrive/Documentos/Faculdade Senac/Git Repositor/.git/

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor (master)
$ ls -a
./  ../  .git/  teste.txt

armoc@Armoc MINGW64 ~/OneDrive/Documentos/Faculdade Senac/Git Repositor (master)
$ git config --global init.defaultBranch main
```


Ciclo de vida do arquivo



Quatro estados....

Ciclo de vida do arquivo



Repositório

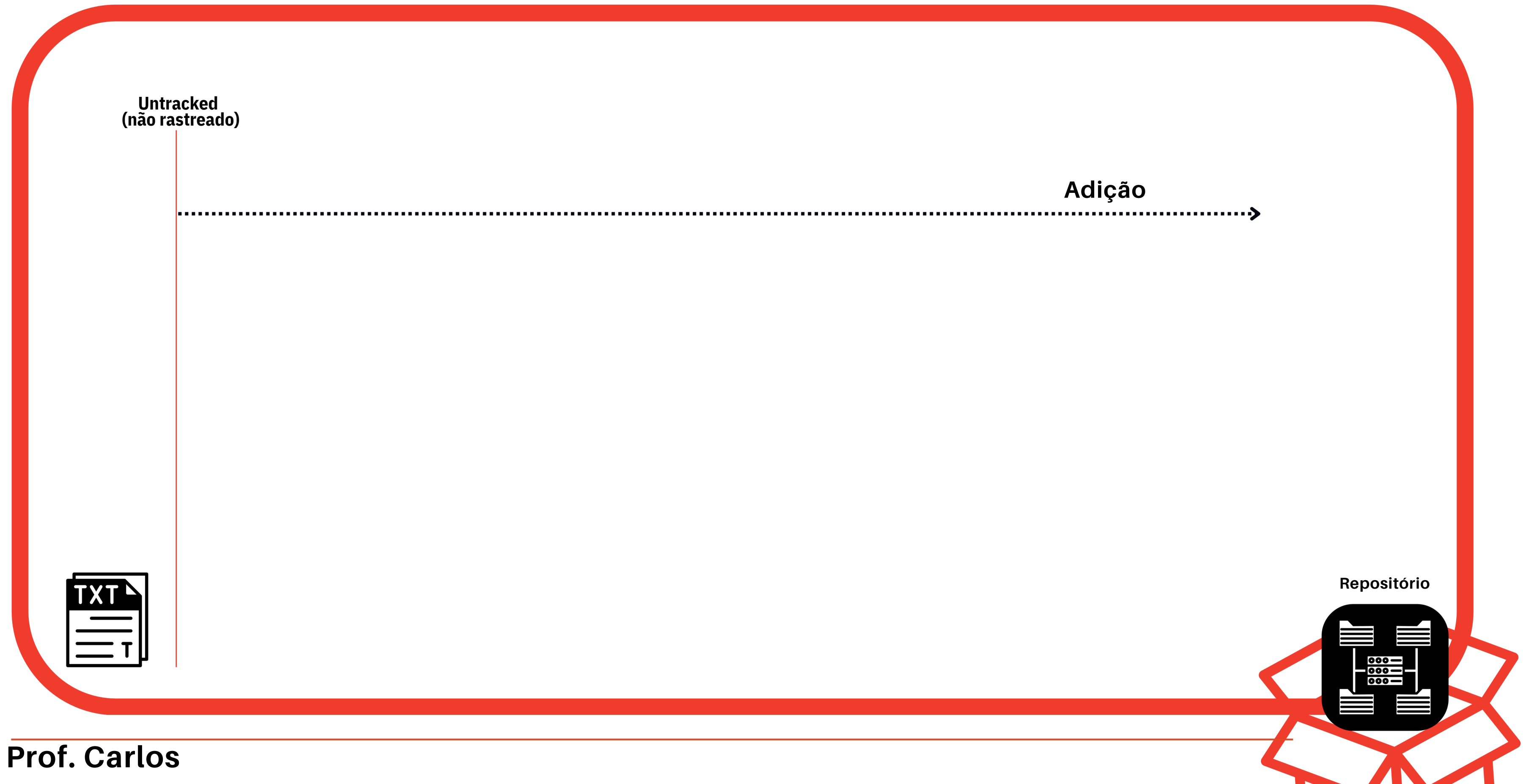


Quatro estados....

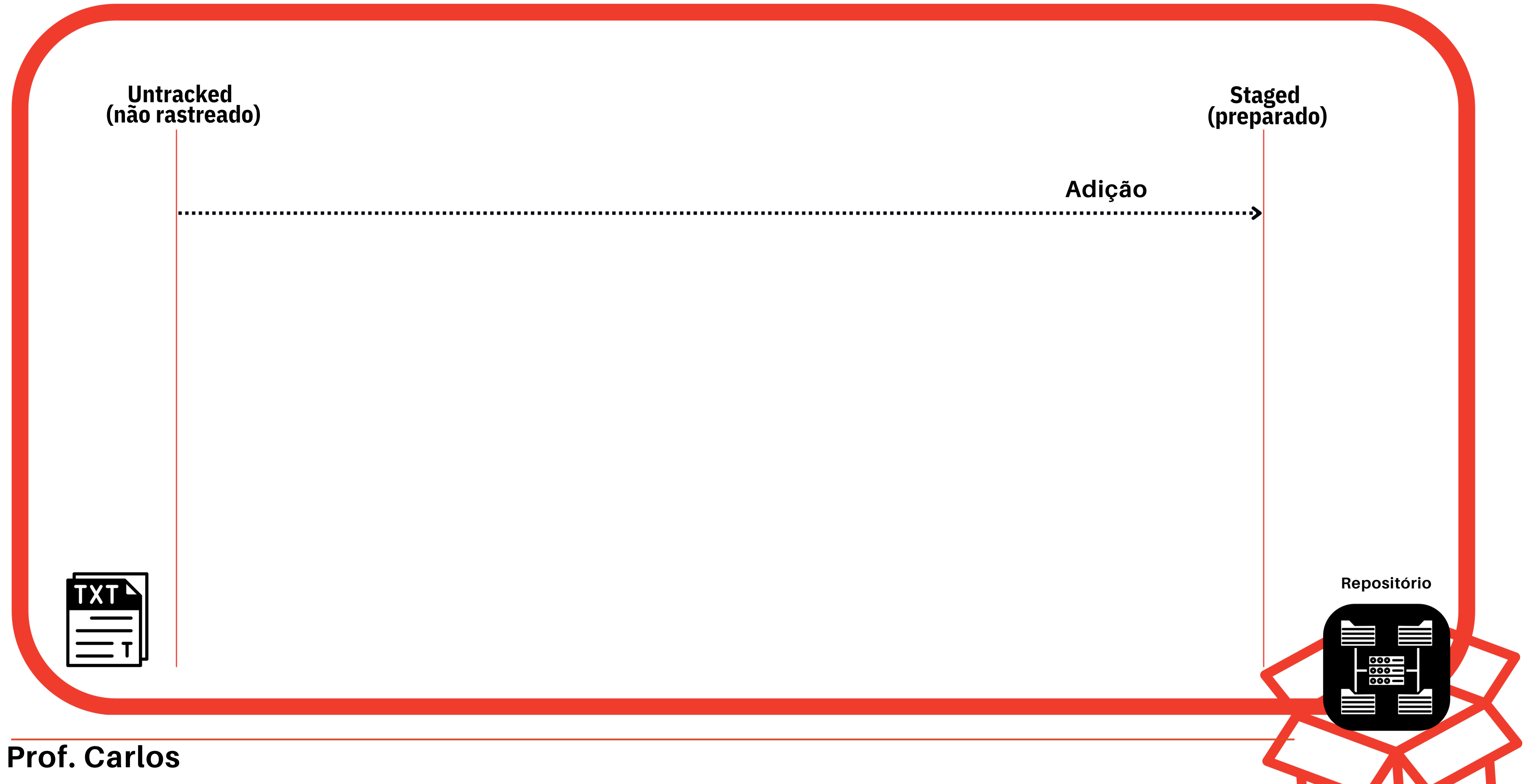
Ciclo de vida do arquivo



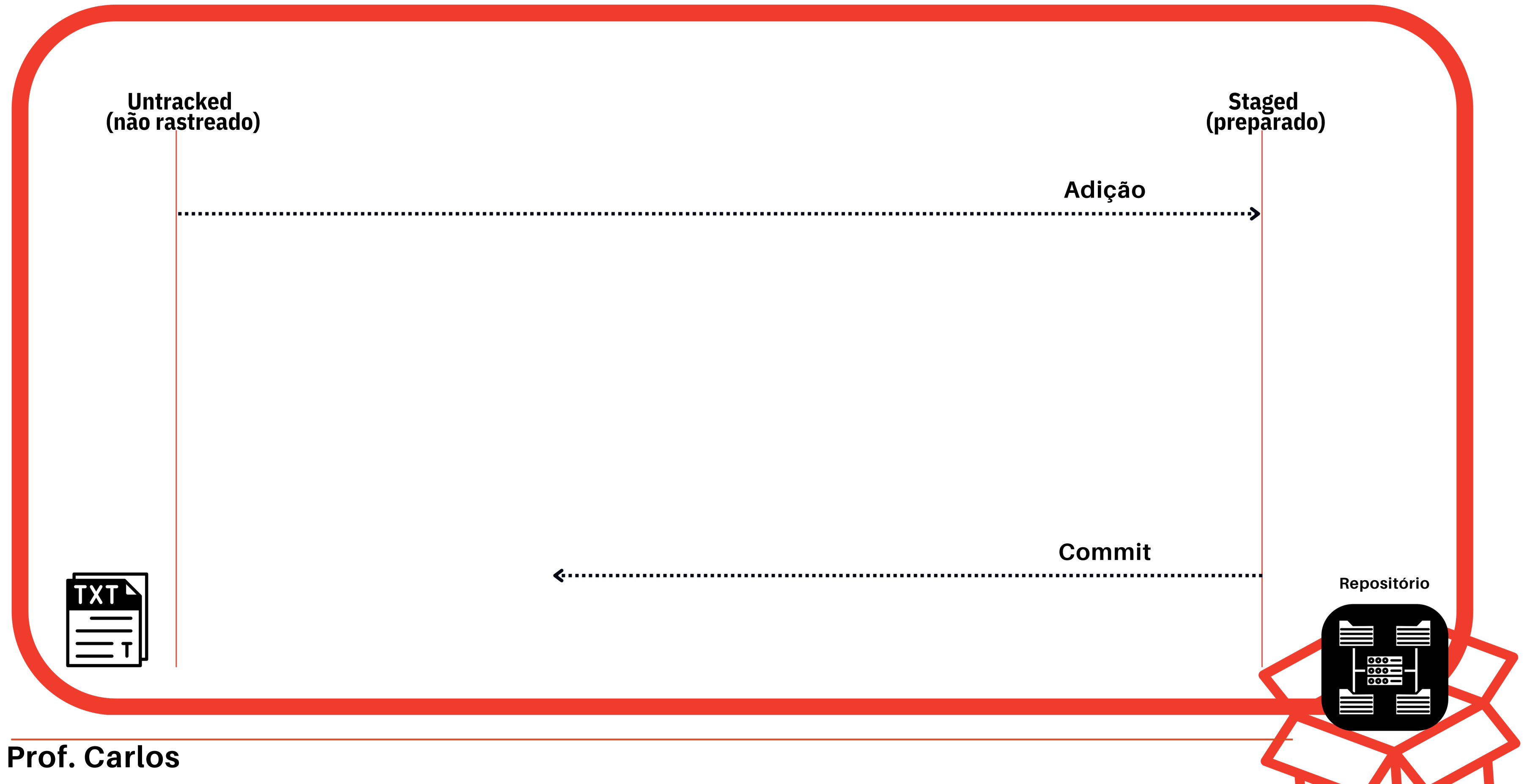
Ciclo de vida do arquivo



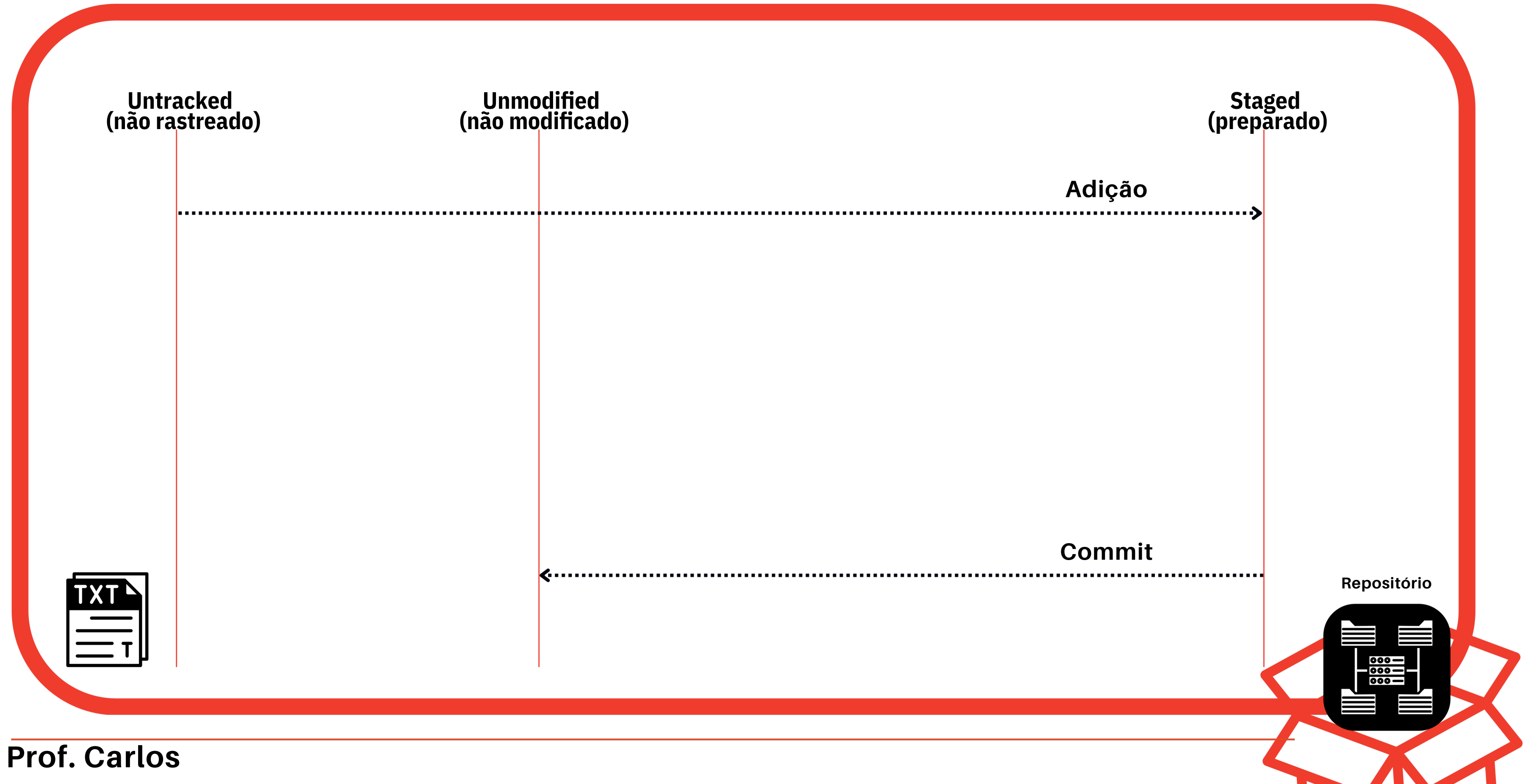
Ciclo de vida do arquivo



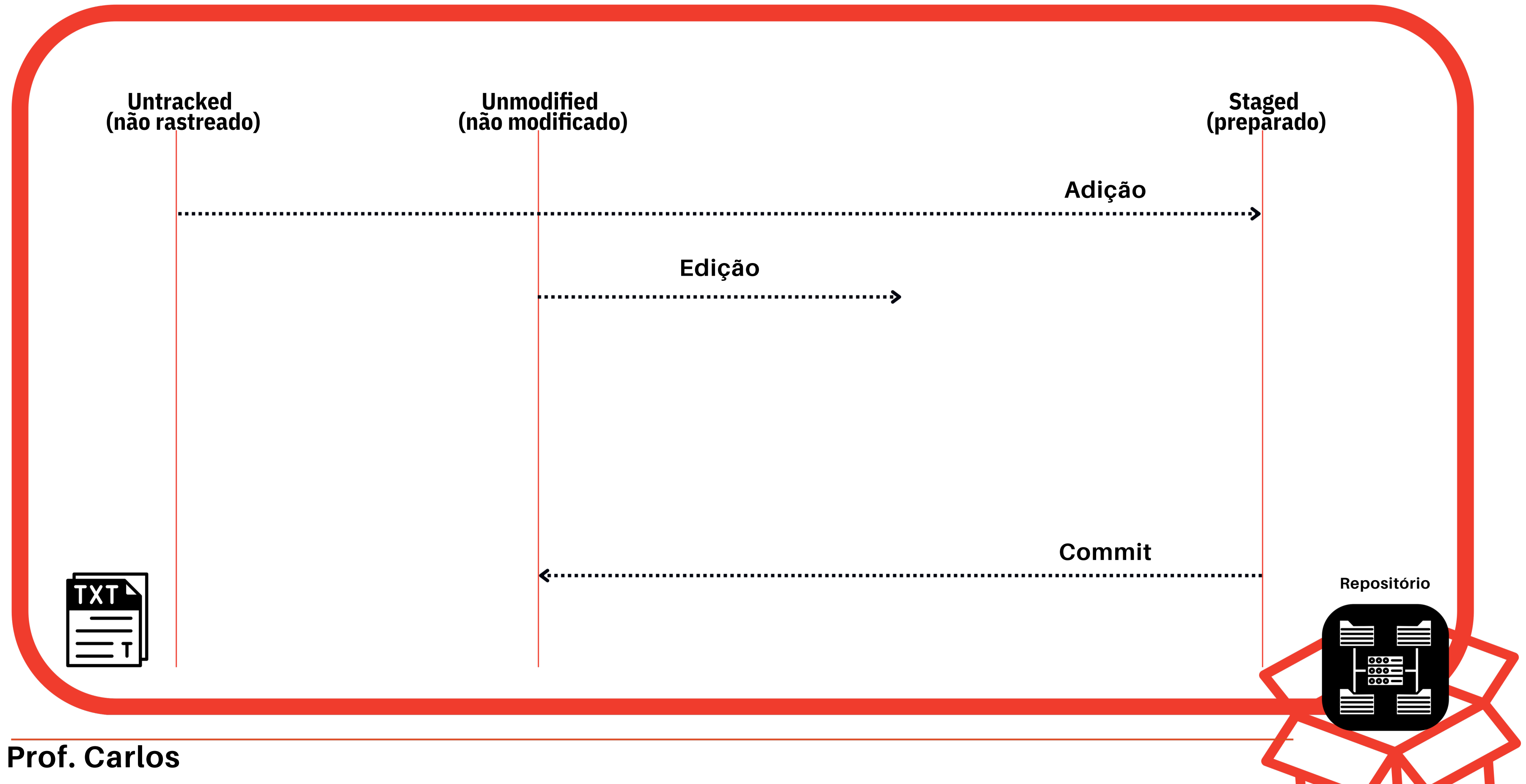
Ciclo de vida do arquivo



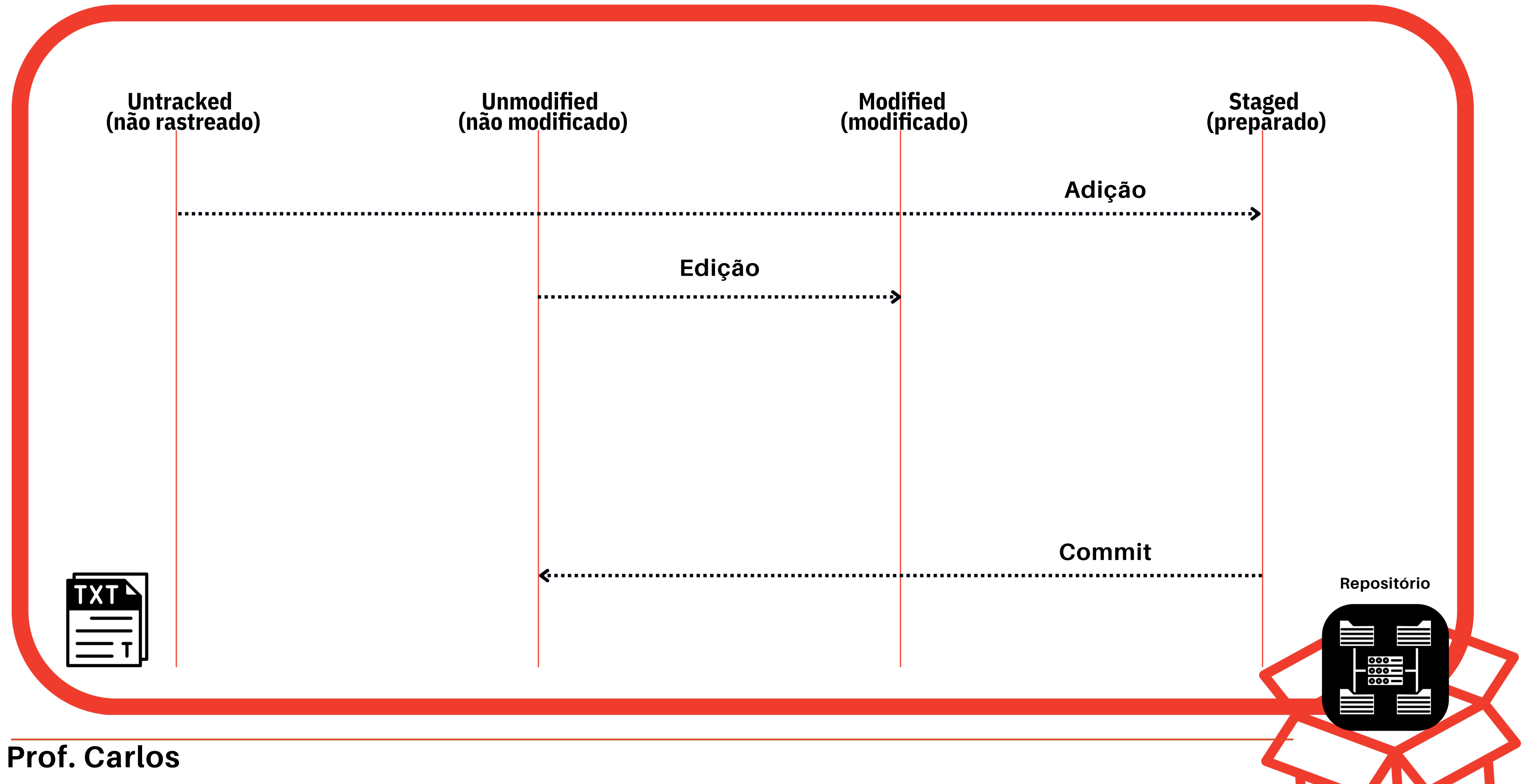
Ciclo de vida do arquivo



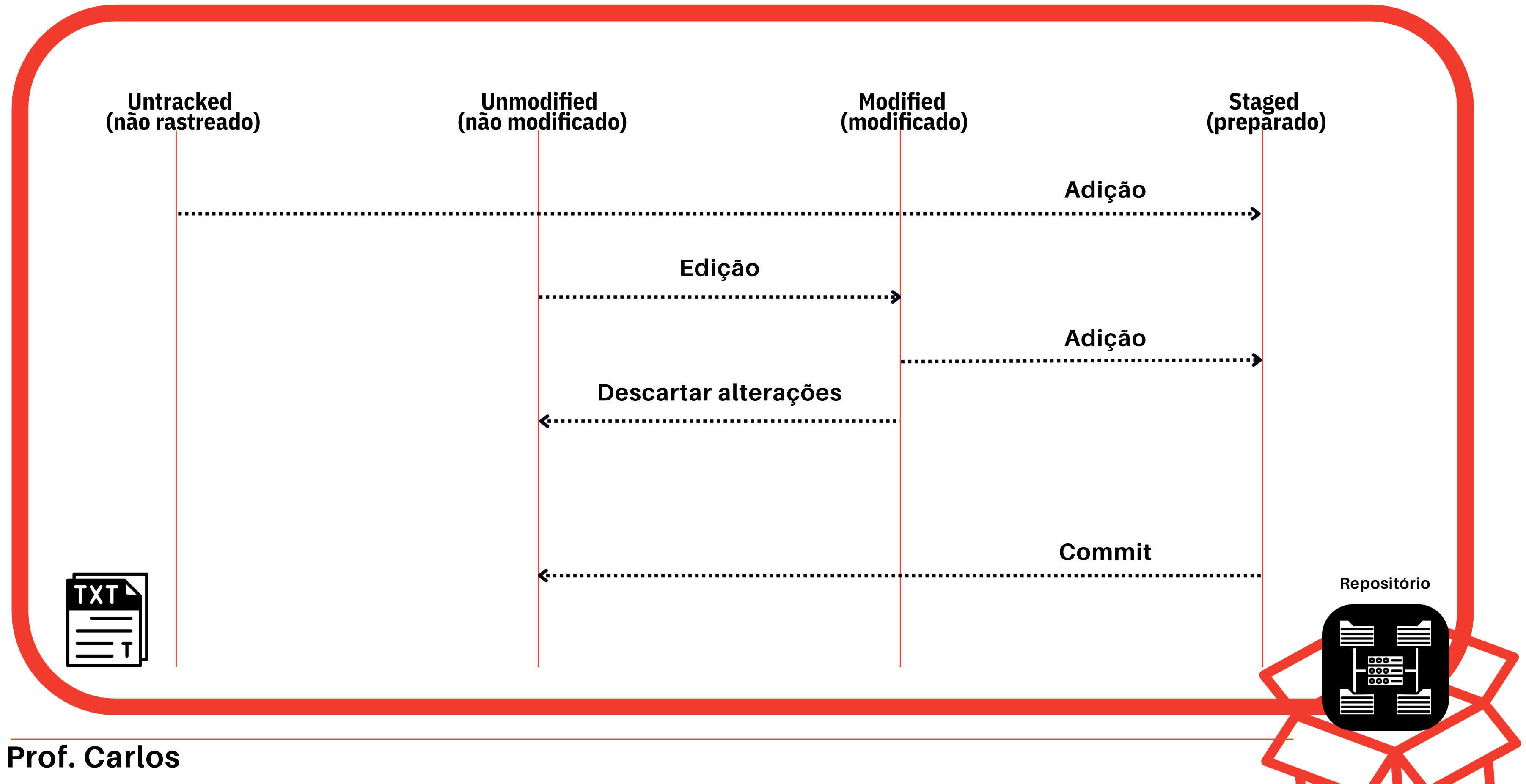
Ciclo de vida do arquivo



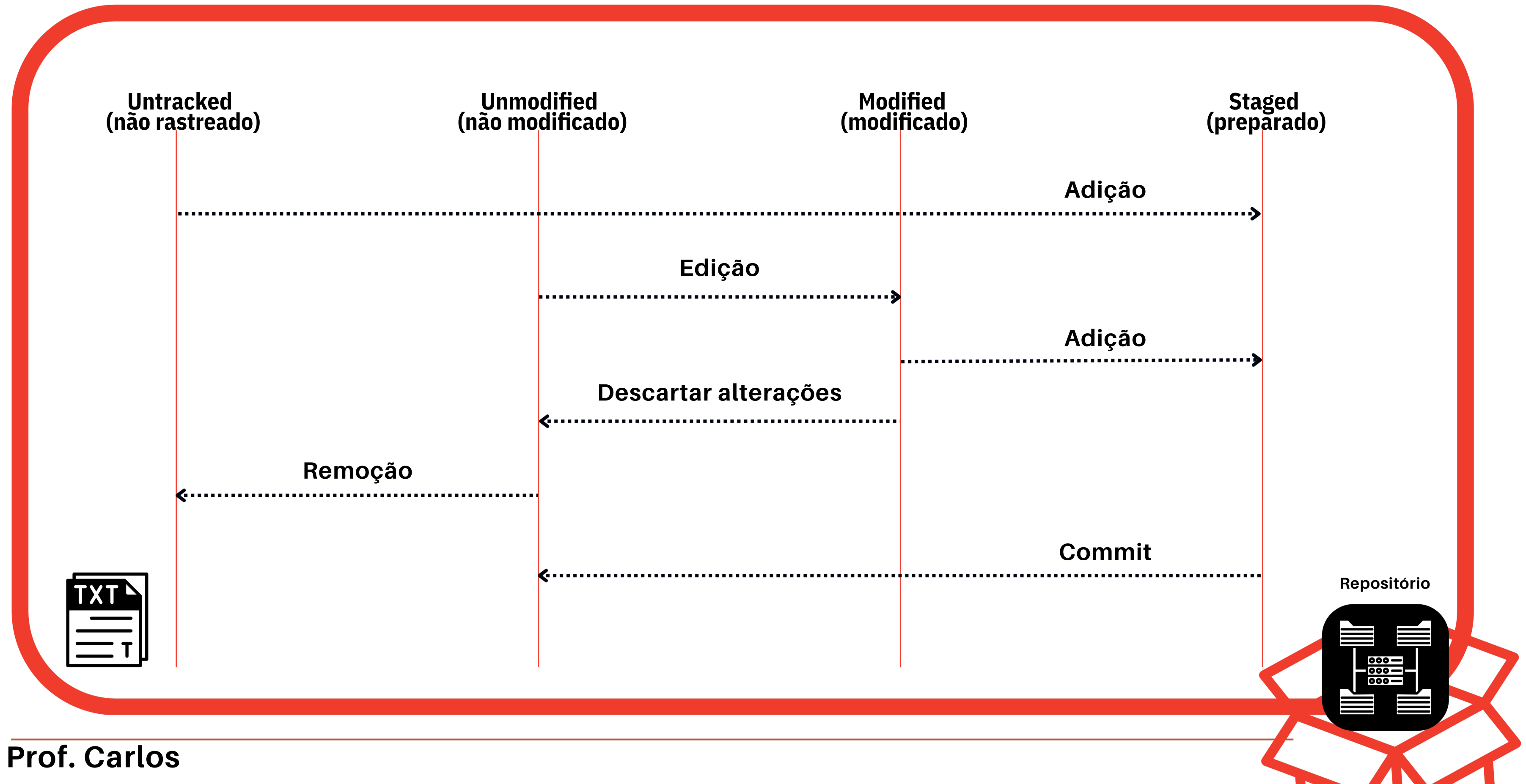
Ciclo de vida do arquivo



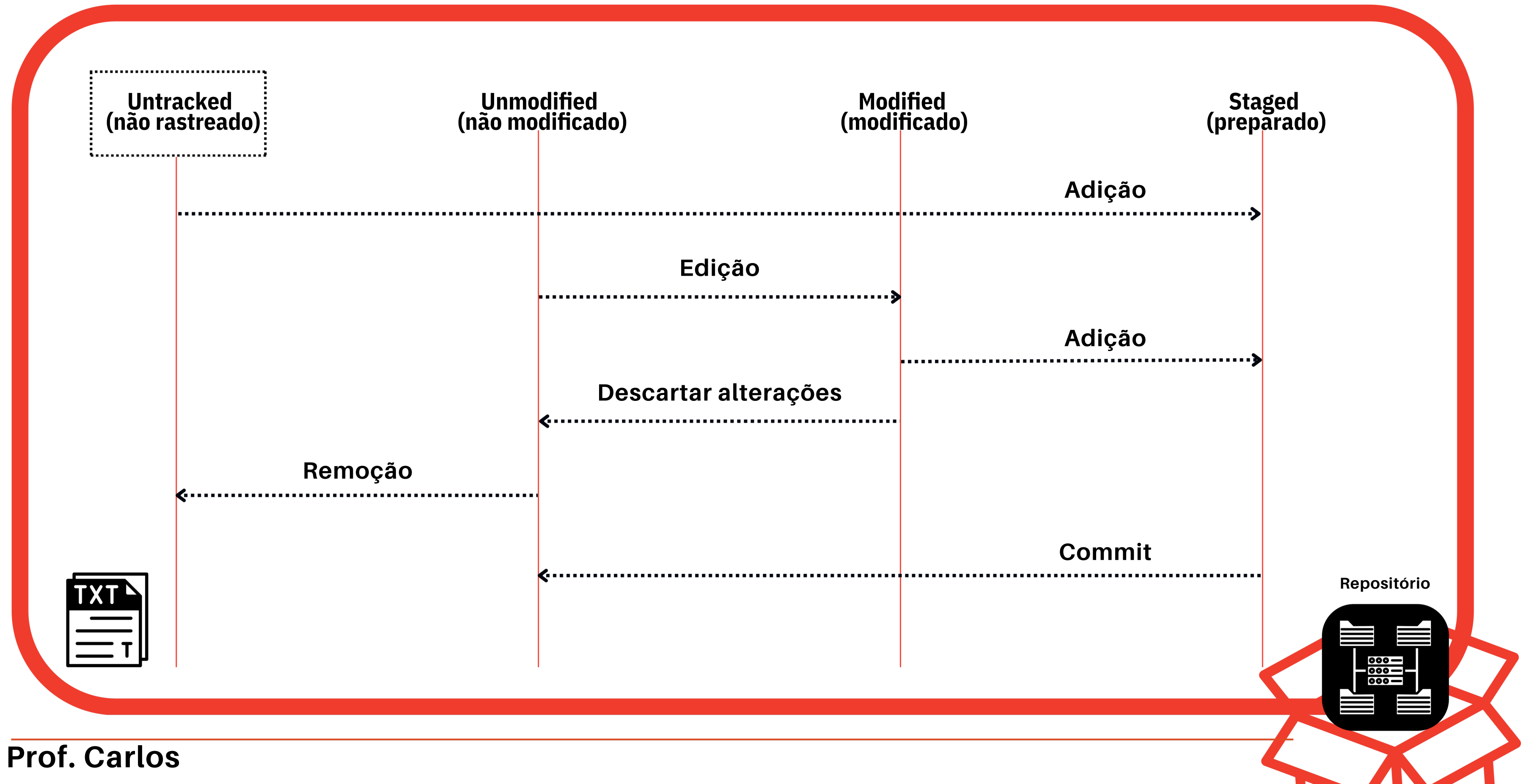
Ciclo de vida do arquivo



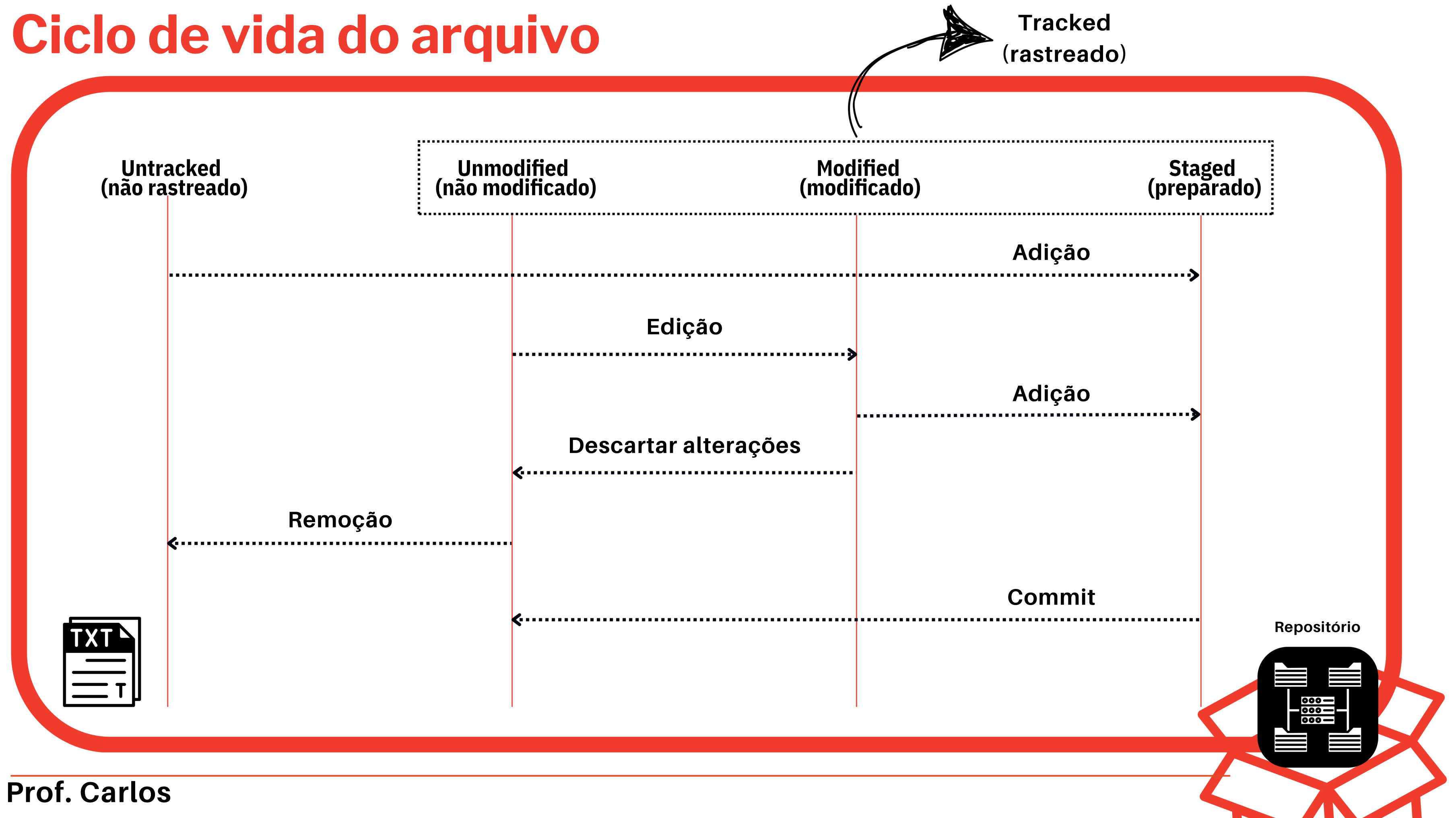
Ciclo de vida do arquivo



Ciclo de vida do arquivo



Ciclo de vida do arquivo



Git Resumo Prático



CONFIGURAÇÃO

<code>git config --global user.name "nome_do_usuario"</code>	Faz a configuração do nome do usuário que será usado nos commits.
<code>git config --global user.email "email_do_usuario"</code>	Faz a configuração do email do usuário que será usado nos commits.
<code>git config --list</code>	Listagem de configurações e valores Git.

COMANDOS DIÁRIOS

<code>git init</code>	Inicializa um repositório Git.
<code>git status</code>	Exibe o estado atual do repositório, mostrando arquivos modificados, preparados para commit e não rastreados.
<code>git add <arquivo></code>	Adiciona um arquivo na área preparação, preparando-o para commit.
<code>git add .</code>	Adiciona todos os arquivos na área preparação, preparando-o para commit.
<code>git commit -m "mensagem"</code>	Registra as mudanças adicionadas ao repositório com uma mensagem.

INSPEÇÃO DE ALTERAÇÕES

<code>git diff</code>	Exibe as diferenças entre o diretório de trabalho e o último commit.
<code>git diff --cached</code>	Mostra diferenças entre arquivos na área de preparação e o último commit.
<code>git diff --staged</code>	Igual ao de cima. Mostra diferenças entre arquivos na área de preparação e o último commit.

REMOÇÃO E RESTAURAÇÃO

<code>git rm --cached -r .</code>	Remove arquivos e diretórios da área de preparação, mantendo-os no diretório de trabalho.
<code>git restore <arquivo></code>	Restaura um arquivo modificado para a versão do último commit.
<code>git restore --stage <arquivo></code>	Retira um arquivo do staging sem perder as alterações.
<code>git checkout <arquivo></code>	Restaura um arquivo para a versão do último commit.
<code>git clean -f</code>	Remove arquivos não rastreados do diretório de trabalho.
<code>git reset --hard</code>	Descarta todas as alterações locais e volta ao último commit.

HISTÓRICO

<code>git log</code>	Mostra o histórico de commits.
<code>git log --oneline</code>	Exibe um resumo dos commits (um por linha).
<code>git log --patched</code>	Mostra detalhes das mudanças feitas em cada commit.
<code>git log --stat</code>	Exibe estatísticas das alterações realizadas.

Git Resumo Prático



ALTERAÇÃO DE COMMITS

<code>git commit --amend</code>	Modifica o último commit, permitindo adicionar arquivos ou alterar a mensagem.
<code>git commit --no-edit</code>	Faz um amend sem alterar a mensagem do commit anterior.

REPOSITÓRIOS REMOTOS

<code>git remote -v</code>	Exibe as URLs dos repositórios remotos associados ao repositório local.
<code>git remote add origin <url></code>	Adiciona um repositório remoto com o nome <i>origin</i> e a URL especificada.
<code>git remote set-url origin <url></code>	Modifica a URL de um repositório remoto já existente, no caso, <i>origin</i> .
<code>git push</code>	Envia as alterações locais para o repositório remoto.
<code>git pull</code>	Baixa e mescla alterações do repositório remoto para o repositório local.
<code>git push --force</code>	Força o envio das alterações para o repositório remoto, sobrescrevendo o histórico de commits no remoto.
<code>git push --force-with-lease</code>	Força o envio das alterações, mas verifica se o repositório remoto não foi alterado antes de sobrescrever o histórico.

MESCLAGEM DE BRANCHES

<code>git merge <branch></code>	Mescla as alterações de uma branch especificada para a branch atual.
---------------------------------------	--

BRANCHES

<code>git branch --list</code>	Exibe a lista de branches locais no repositório.
<code>git branch <nova_branch></code>	Cria uma nova branch com o nome especificado, sem trocar para ela.
<code>git checkout <branch></code>	Altera para a branch especificada.
<code>git checkout -b <nova_branch></code>	Cria uma nova branch e já muda para ela.
<code>git checkout -f <branch></code>	Força a troca para a branch especificada, descartando alterações não commitadas.
<code>git switch <branch></code>	Alterna para a branch especificada, alternativa ao <i>git checkout</i> .
<code>git switch -</code>	Volta para a última branch onde estava.
<code>git switch -c <nova_branch></code>	Cria uma nova branch e já muda para ela, equivalente ao <i>git checkout -b</i> .
<code>git branch -d <nome_branch></code>	Exclui a branch local especificada, se ela já tiver sido mesclada.
<code>git push --delete origin <nome_branch></code>	Remove a branch especificada do repositório remoto.
<code>git branch -m <novo_nome></code>	Renomeia a branch atual para o novo nome.
<code>git branch -m <nome_antigo> <novo_nome></code>	Renomeia a branch especificada de <nome_antigo> para <novo_nome>.
<code>git branch -a</code>	Exibe todas as branches, locais e remotas.

Git Resumo Prático



TAGS

<code>git tag <nome></code>	Cria uma tag do tipo lightweight (sem metadados), associada ao último commit.
<code>git tag -a -m "mensagem" <nome></code>	Cria uma tag annotated (com metadados) e adiciona uma mensagem de anotação.
<code>git show <tag></code>	Exibe informações sobre uma tag específica, incluindo o commit ao qual está associada.
<code>git tag --list</code>	Exibe todas as tags presentes no repositório.
<code>git tag -n</code>	Exibe todas as tags com suas respectivas mensagens (se houver).
<code>git tag -d <nome></code>	Remove uma tag localmente.
<code>git push --delete origin <nome_tag></code>	Remove uma tag do repositório remoto.

MODIFICAÇÃO DE HISTÓRICO

<code>git revert <commit></code>	Cria um novocommit que desfaz as alterações de um commit anterior.
<code>git revert HEAD --no-edit</code>	Reverte o último commit sem modificar a mensagem de commit.
<code>git rebase <branch></code>	Reaplica commits da branch atual sobre a branch especificada.
<code>git rebase --abort</code>	Aborta o processo de rebase em andamento, retornando ao estado anterior.
<code>git rebase --continue</code>	Continua o rebase após a resolução de conflitos.

STASH

<code>git stash</code>	Armazena temporariamente alterações não commitadas, deixando o diretório de trabalho limpo.
<code>git stash list</code>	Exibe a lista de stashes armazenados.
<code>git stash apply</code>	Aplica o stash mais recente ao diretório de trabalho, sem removê-lo da lista de stashes.
<code>git stash apply stash@<posicao></code>	Aplica um stash específico da lista, indicado pela posição.
<code>git stash pop <posicao></code>	Aplica e remove o stash especificado da lista de stashes.
<code>git stash drop <posicao></code>	Remove um stash específico da lista, sem aplicá-lo.

BISECT (bissecção)

<code>git bisect start</code>	Inicia uma sessão de bisecting para identificar o commit que introduziu um erro.
<code>git bisect good <hash></code>	Marca o commit especificado como "bom" durante a bissecção.
<code>git bisect bad <hash></code>	Marca o commit especificado como "ruim" durante a bissecção.
<code>git bisect good</code>	Marca o commit atual como "bom" no processo de bissecção.
<code>git bisect bad</code>	Marca o commit atual como "ruim" no processo de bissecção.

Git Resumo Prático



FETCH

<code>git fetch</code>	Baixa as atualizações de repositórios remotos, mas não as mescla com a branch atual.
------------------------	--

APLICAÇÃO DE COMMIT

<code>git cherry-pick <commit></code>	Aplica as mudanças de um commit específico da outra branch à branch atual.
---	--

ALIAS

<code>git config --global alias.<abreviação> <comando></code>	Cria um alias global, permitindo usar <abreviação> no lugar de <comando>.
<code>git config --global --unset alias.<abreviacao></code>	Remove o alias global especificado.

